

The Federated Enterprise Agent Platform

Architecture, adversarial design rationale, runtime guardrails and capacity engineering

This document argues for a specific shape of enterprise agent platform, one where independently owned domain agents are composed by a deliberately thin central runtime, where authority is deterministic rather than inferred, and where the fast path runs on quantized models inside the estate while frontier reasoning is reserved for the small fraction of turns that genuinely need it.

It is written adversarially. Each competing design is given its strongest form before it is criticised, every claim is paired with the observation that would falsify it, and one chapter is spent entirely on where this architecture is worse than the alternatives.

AUTHOR	Sarang Pramode
DOCUMENT	Version 1.0
DATE	August 2026
STATUS	Technical design document, for review

All hardware, latency and throughput figures in this document are third-party published values or explicitly modelled estimates. They are starting points for capacity planning, not guarantees. Chapter 23 gives the exact commands to re-measure every one of them on your own workload.

Contents

Part I - Position	6
1. Executive Summary	6
2. The Problem, Stated Precisely Enough to Argue With	9
2.1 What the enterprise actually has	9
2.2 What the enterprise wants	9
2.3 The three forces that decide the architecture	9
3. Scope, Audience and How to Read This	11
3.1 Audience	11
3.2 What is in scope	11
3.3 What is deliberately out of scope	11
3.4 A note on the word "customer"	11
3.5 How to read this document	11
4. The Reference Implementation	12
4.1 What exists and what it is for	12
4.2 The twelve steps and the claims they make falsifiable	12
4.3 The auditor, and why it is not a model	14
4.4 What the prototype does not yet do	15
Part II - Why This Design, Adversarially	16
5. How to Argue About Architecture	16
5.1 The criteria	16
5.2 On premature decomposition	17
6. Alternative A: The Single Agent With Skills	17
6.1 The design in its strongest form	17
6.2 Mechanism one: change surface grows with the catalogue, and it grows in a shared mutable global	18
6.3 Mechanism two: prefill cost grows with capability count, on every turn, whether or not the capabilities are relevant	
6.4 Mechanism three: prompt shadowing, the failure that domain evaluation cannot see	19

6.5 Mechanism four: every domain release becomes a system release	20
6.6 What would falsify this criticism	20
7. Alternative B: Peer to Peer, Without a Centre	20
7.1 The design in its strongest form	20
7.2 Mechanism one: nobody owns the end-to-end deadline	20
7.3 Mechanism two: no closed-world reachability, therefore no computable blast radius	21
7.4 Mechanism three: trust and compatibility edges grow quadratically	21
7.5 Mechanism four: delegation loops are emergent rather than designed	21
7.6 Mechanism five: no trace root, so attribution becomes archaeology	22
7.7 What would falsify this criticism	22
8. Alternative C: The Monolith Repository	22
8.1 The design in its strongest form	22
8.2 Mechanism one: release coupling compounds with team count	22
8.3 Mechanism two: continuous integration wall clock grows with total capability	23
8.4 Mechanism three: one dependency vulnerability forces a full-platform redeploy	23
8.5 Mechanism four: no independent scaling, and no independent failure	23
8.6 What would falsify this criticism	24
9. Alternative D: Federated, But Ungoverned	25
9.1 The design in its strongest form	25
9.2 Mechanism one: evidence selected by the changing team is not independent evidence	25
9.3 Mechanism two: declared capability drifts from actual capability	25
9.4 Mechanism three: nothing establishes that evidence was gathered against the artefact being released	26
9.5 What would falsify this criticism	26
10. Alternative E: In-Process Orchestration Versus Service Federation	26
10.1 The design in its strongest form	26
10.2 Mechanism one: modularity without fault isolation	26
10.3 Mechanism two: one dependency graph, one runtime, one version	27
10.4 Mechanism three: no independent deployment, so no independent cadence	27
10.5 Mechanism four: no per-domain resource governance	27

10.6 What would falsify this criticism	27
11. Where This Design Is Worse	27
11.1 It adds latency to every request, forever	27
11.2 Debugging is genuinely harder, and the tooling debt comes first	27
11.3 There is a platform team headcount floor, and it does not amortise at small scale	28
11.4 The control plane is a new correlated failure mode	28
11.5 Contract and versioning overhead is a real tax on small changes	28
11.6 Evaluation cost is partitioned but not reduced, and coordination cost is added	28
11.7 Cognitive load rises for everyone, including people who did not ask for it	28
11.8 Summary of when not to adopt this design	29
12. The Comparison, Quantified	29
 Part III - Runtime Design	 31
13. Anatomy of the Central Agent	31
13.1 Five responsibilities, and the refusal of a sixth	31
13.2 Stage by stage, with a budget	31
13.3 What the centre must never own	33
13.4 Statelessness, sessions and the durable boundary	34
13.5 The single chokepoint, and why it is worth the risk	34
14. The Model Tiering Ladder	35
14.1 Why tiering rather than one good model	35
14.2 The four tiers	37
14.3 The escalation policy	37
14.4 Confidence gating, and what a margin actually means	38
14.5 The cost arithmetic that justifies the ladder	38
14.6 Operating the ladder	38
15. On-Prem Quantized Versus Frontier API	39
15.1 The question, stated so that it can be answered	39
15.2 The same journey, twice	39

15.3 Why the API penalty is structural rather than incidental	41
15.4 The second-order consequences	41
15.5 The crossover, computed	42
15.6 Where frontier models genuinely win	42
15.7 What would falsify this chapter	43
16. Guardrails: Taxonomy and Placement	43
16.1 What a guardrail is, and what it is not	43
16.2 The four boundaries where rails attach	43
16.3 The input rail taxonomy	45
16.4 Injection usually does not arrive in the user's message	45
16.5 The output rail taxonomy	46
16.6 Fail open or fail closed, decided per control	46
17. The Guardrail Cascade	47
17.1 The guardrail tax, and why chaining safety calls does not work	47
17.2 The cascade	48
17.3 Model choices at each stage	49
17.4 The rails need their own evaluation	50
17.5 Who owns the thresholds	50

PART I

Position

Part I states the claim, defines the problem precisely enough to be argued with, and grounds the rest of the document in a working prototype whose artefacts are named throughout.

1. Executive Summary

An enterprise that wants conversational and agentic access to its own capabilities faces a structural problem long before it faces a modelling problem. The capabilities live in different systems, are owned by different teams, change on different release cadences, carry different regulatory obligations, and are correct according to rules that only their owning team can state authoritatively. Any architecture that requires those teams to converge on a single prompt, a single deployable, or a single release train is not solving the enterprise's problem; it is asking the enterprise to reorganise itself around a model.

This document argues for federation with a thin centre. Each business domain publishes a versioned agent that owns its own prompts, skills, tools, evaluation rubrics and deployment lifecycle. A central runtime, owned by a platform team, does five things and refuses to do more: it classifies the request, discovers which declared capabilities are eligible, plans the smallest sufficient sequence of calls, enforces authority deterministically, and composes a response that has passed output validation. Everything else, including the semantics of any business rule, belongs to a domain.

Three design commitments distinguish this from the general idea of "multi-agent".

The first is that **discovery is closed-world and independently audited**. A capability is reachable if and only if it appears in a published, hash-identified bundle, and an auditor that is not the changing team recomputes that property on every change. In the reference implementation this is the `reachability-closed` check, a set comparison whose failure detail reads "Central can reach a surface that is not identical to the published bundle." This is what makes targeted evaluation sound rather than merely cheap: the blast radius of a change is a computable set, not an act of judgement by the person who made the change.

The second is that **the model never holds authority**. Entitlement, limits, idempotency and write permission are decided by deterministic code reading structured state. A model proposes; a policy engine disposes. This is not a stylistic preference. It is the only way to answer a regulator's question about why a specific customer was allowed to perform a specific action with a reproducible artefact rather than a probability distribution.

The third is that **model capability is tiered, and the fast path is local**. Roughly two thirds of production turns should be resolvable with no model at all. Most of the remainder need a small quantized model, served inside the estate, answering in tens of milliseconds. Frontier reasoning is reserved for cross-domain decomposition, contested policy interpretation and plan repair, which together should account for a few percent of turns. Chapter 15 works the counterfactual in detail: routing the same journey's control path through a frontier API instead costs roughly 458 ms versus 2,115 ms end to end on identical domain work, a factor of about 4.6, and buys a set of second-order liabilities that latency alone does not reveal, including shared provider quota

coupling user traffic to evaluation traffic, and raw utterances leaving the estate on exactly the hop where capability was gained least.

The document then does the work that architecture papers usually skip. Part II gives each rejected alternative its strongest form and states the specific mechanism by which it fails at scale, along with the observation that would prove the criticism wrong. Chapter 11 is an honest accounting of what this design costs: additional network hops on every journey, a genuinely harder debugging story, a platform team headcount floor that does not amortise below a certain scale, and a new control-plane dependency that becomes a correlated failure mode. Part IV converts a business number into a GPU count through an explicit chain of assumptions, sizes three deployment tiers, and states the accuracy price of each quantization format rather than hiding it. Part V covers assurance, and Part VI is the practical residue: anti-patterns, a phased roadmap, and a question bank for design review.

The intended outcome is not agreement. It is that a technical audience can locate precisely where they disagree, and know what measurement would settle it.

Design rule. Prefer the fewest reasoning hops that still preserve domain autonomy, enforceable authority and independent deployability. Every additional hop must be justified by a capability that cannot be obtained more cheaply, and every model call must be justified by a decision that deterministic code cannot make.

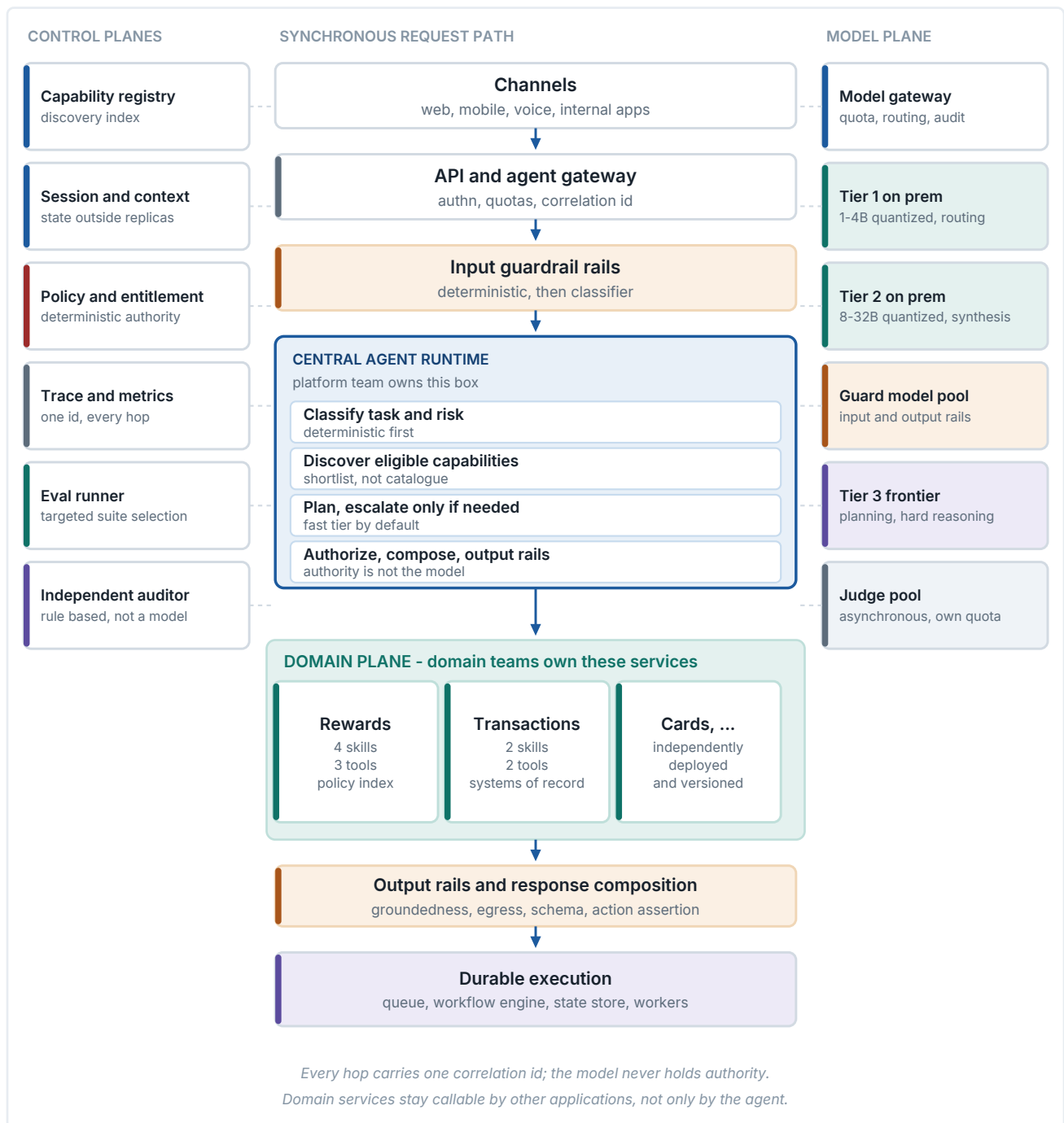


Figure 1. The reference architecture. The synchronous request path is deliberately short and owned end to end by the platform team. Control planes on the left are what make federation governable rather than merely distributed. The model plane on the right is tiered, with the fast path resident in the estate and frontier capacity reserved for escalation. Note what is absent: no path by which one domain calls another, and no point at which a model grants itself authority.

2. The Problem, Stated Precisely Enough to Argue With

2.1 What the enterprise actually has

Before discussing agents it is worth being concrete about the substrate. A large financial services, telecommunications or retail organisation typically has between thirty and several hundred business capabilities that a customer might reasonably ask about. Each is implemented across systems of record of varying age, wrapped in services of varying quality, and governed by rules that are partly in code, partly in policy documents, and partly in the heads of the four people who have owned the domain for a decade.

Three properties of this substrate drive the architecture, and all three are usually understated in agent design discussions.

Ownership is real and it is not going away. The team that owns rewards accrual is accountable for whether a customer is told the correct multiplier. That accountability is contractual, sometimes regulatory, and it does not transfer to a platform team merely because the platform team now operates the interface through which the customer asks. Any design that separates the accountability for an answer from the ability to change how that answer is produced will fail, not technically but organisationally, and the failure will present as a queue of unresolved tickets rather than as an outage.

Cadence is heterogeneous by necessity. A payments domain under a scheme mandate may ship weekly with heavy change control. A marketing offers domain may want to ship several times a day. A core banking integration may ship quarterly because its downstream dependency does. Forcing these onto a common release train does not make the slow ones faster; it makes the fast ones slow and creates an incentive to bypass the platform entirely.

Correctness is domain-local and often surprising. The reference implementation encodes a deliberately small example of this: restaurant purchases earn a 3x multiplier and grocery purchases earn 2x, so a \$100 restaurant transaction earns 300 points. That rule is trivially simple and yet an incorrect prompt reduced it to the grocery multiplier and produced a confidently worded answer of 200 points. Nothing about that answer was malformed, unsafe, off-topic or ungrounded in the retrieved transaction. It was simply wrong in a way that only the rewards domain could adjudicate. Multiply that by a hundred domains, each with rules an order of magnitude more intricate, and the futility of centralising semantic correctness becomes clear.

2.2 What the enterprise wants

Stated as jobs to be done rather than as features, the platform must let a customer or employee express an intent in natural language and receive an answer that is correct according to the owning domain, permitted according to policy, attributable to a specific chain of decisions, and delivered inside a latency budget that keeps the interaction conversational. It must let a domain team add or correct capability without coordinating with any other team. It must let a governance function answer, after the fact and without the implementation team's cooperation, what the system was permitted to do at a given moment and why a specific decision was taken. And it must let a finance function attribute cost to journeys rather than to an undifferentiated model bill.

These are ordinary distributed-systems requirements. That observation is the point of this document. The novelty of language models has encouraged a great deal of architecture that would be rejected immediately in any other domain, and the discipline this design applies is mostly the recovery of ordinary practice: explicit contracts, bounded blast radius, deterministic authority, independent verification, per-dependency timeouts, and capacity planning from measured load.

2.3 The three forces that decide the architecture

Every design decision in this document can be traced to a tension between three forces, and most disagreements about agent architecture are really disagreements about their relative weight.

Latency is a product constraint, not an engineering preference. Conversational interaction degrades sharply once the first token is delayed beyond roughly a second, and abandonment rises steeply after that. This is a hard budget that must be divided among identity, guardrails, routing, domain work, validation and composition. Chapter 13 divides it explicitly. The consequence that most architectures resist is that model calls on the critical path are expensive in the currency that matters most, so the number of them must be minimised rather than merely optimised.

Change safety is the dominant long-run cost. A platform that cannot absorb a domain change without a full-system regression will either accumulate an unaffordable evaluation bill or stop evaluating properly. Chapter 26 develops this, but the structural conclusion appears early: the architecture must make the blast radius of a change computable, because only then can evaluation be targeted without being negligent.

Autonomy is what makes the platform adoptable. If joining the platform costs a domain team its release independence, the platform's growth is capped by the platform team's capacity, which is precisely the bottleneck federation exists to remove.

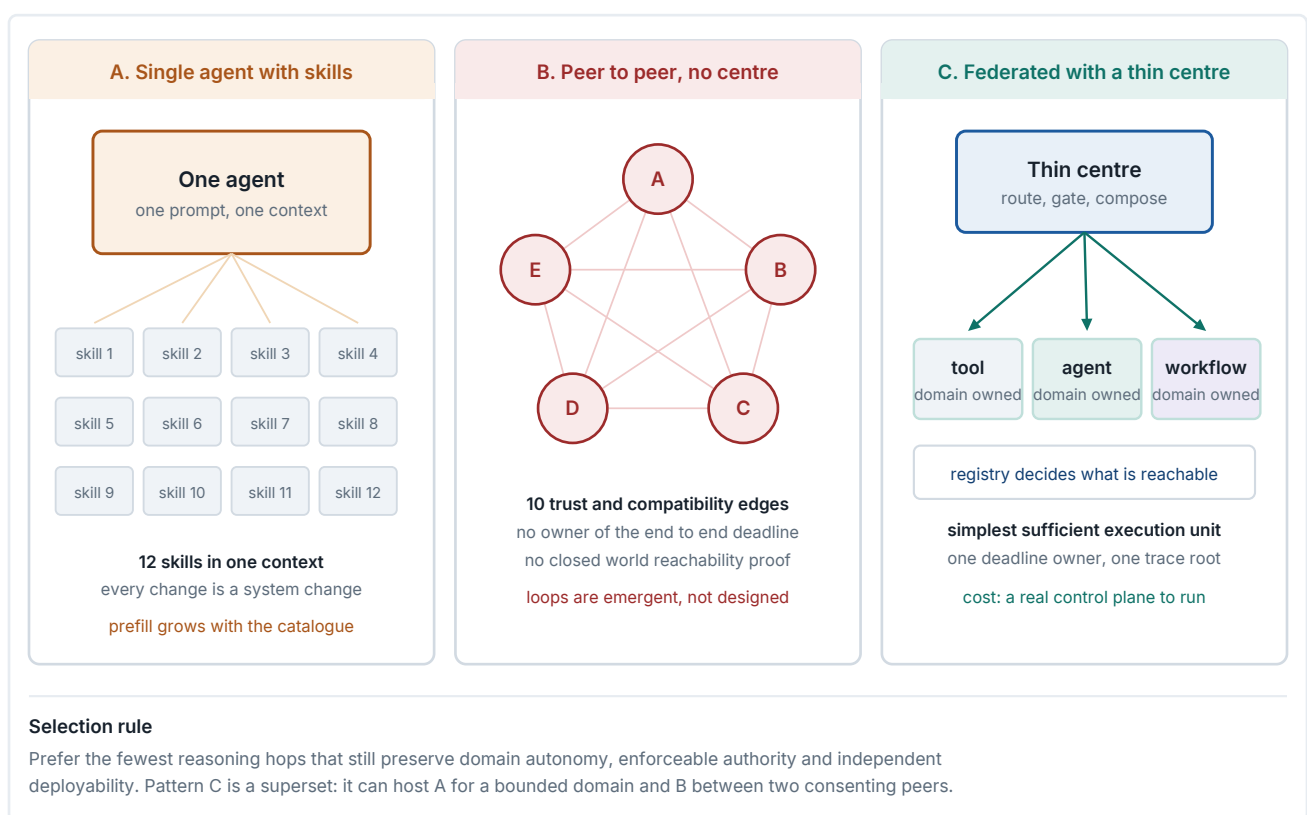


Figure 2. The three candidate shapes, with the mechanism of failure named in each panel rather than left implicit. Panel C is a superset of the others: a bounded domain may legitimately be implemented as panel A behind its agent card, and two consenting peers may exchange messages directly if their contract says so. What panel C adds is an owner of the end-to-end deadline, a single trace root, and an artefact stating what the system is permitted to do.

3. Scope, Audience and How to Read This

3.1 Audience

This document is written for the people who will be asked to sign off on the design or operate it: enterprise and solution architects, platform and ML infrastructure engineers, heads of engineering deciding whether to fund a platform team, and risk, security and governance functions who need to know where their controls attach. It assumes familiarity with distributed systems and with the practical behaviour of language models, and it does not explain what a transformer is.

3.2 What is in scope

The architecture of the request path and the control planes around it; the division of responsibility between the centre and the domains; the placement, layering and evaluation of input and output guardrails; the model tiering strategy and the escalation policy that governs it; hardware sizing from a business load figure through to a GPU count; the engineering capabilities required to build and keep the thing running; and the assurance apparatus that makes any of it defensible.

3.3 What is deliberately out of scope

Vendor selection, procurement and commercial terms. Specific framework choices, which change faster than this document could track and which the architecture is designed to survive. Model pre-training and the construction of foundation models. Data platform architecture beyond the points where the agent platform touches it. And organisation design, except where an architectural choice has an unavoidable organisational consequence, in which case the consequence is stated.

3.4 A note on the word "customer"

The sizing chapters use three load tiers described informally as fewer than ten customers, one thousand to five thousand, and more than fifty thousand. Because the word "customer" is ambiguous in exactly the way that ruins capacity planning, it is pinned down here and used consistently thereafter.

Table 1. How the informal tier labels map onto the quantity that actually determines hardware. Registered user counts are useful for a business case and useless for sizing; the chain in Chapter 18 converts between them explicitly.

Tier label	Informal reading	The quantity that sizes hardware	Approximate registered base implied
Tier S	Fewer than 10 customers	Fewer than about 40 peak concurrent assisted sessions	A pilot, a design partner, or an internal beta group of a few thousand people
Tier M	1,000 to 5,000 concurrent	1,000 to 5,000 peak concurrent assisted sessions	Roughly 0.3 to 2 million registered users, depending entirely on engagement
Tier L	More than 50,000 customers	More than 50,000 peak concurrent assisted sessions	Tens of millions of registered users, or a smaller base with very high engagement

The tier that matters is the middle column. Two organisations with identical registered user counts can differ by an order of magnitude in concurrent sessions, and the ratio between them is the single assumption most worth measuring early.

3.5 How to read this document

Part I establishes the position. Part II is the argument, and is the part to read first if you are inclined to disagree, particularly Chapter 11, which lists the costs of this design without mitigation. Part III is the runtime design and is the most directly implementable material. Part IV is the sizing work and is written to be used as a worksheet rather than read once; Chapter 23 in particular exists so that its own numbers can be replaced with measured ones. Parts V and VI cover assurance and practice.

Every quantitative claim in this document falls into one of three categories, and each is labelled where it appears: measured in the reference implementation, third-party published with a citation, or modelled from stated assumptions. There are no unlabelled numbers, and no figure in this document should be used to buy hardware before Chapter 23 has been executed against the actual workload.

4. The Reference Implementation

4.1 What exists and what it is for

The architecture in this document is accompanied by a working prototype: a Next.js console that walks an audience through twelve steps, each of which exercises one control surface of the platform and produces a named artefact. It is explicitly a visual-first prototype. Guided Demo Mode replays a deterministic event stream through the same reducer and the same components that a live backend would drive, with a mock transport that emits most events 220 ms apart and evaluation or auditor check events 70 ms apart so that a room can follow them. Live Mode is reserved for the phase-two services, which the prototype's configuration anticipates as a central runtime and two domain services alongside the interface.

Being precise about what a prototype proves is a matter of professional hygiene, so it is stated plainly. The prototype demonstrates that the control surfaces exist, that the seams fall in defensible places, and that the platform's claims are expressible as artefacts a reviewer can inspect. It demonstrates nothing whatsoever about throughput, tail latency or cost, because it performs no inference. Every performance number in Part IV is third-party or modelled, and is labelled as such.

4.2 The twelve steps and the claims they make falsifiable

The value of the demonstration is not that it works. It is that each step converts an architectural assertion into something an observer could catch being false.

The twelve demonstration steps, and the architectural claim each one makes falsifiable			
Step	What the demo shows	Artefact in the prototype	The claim it makes testable
1	Central platform only	no domain connected, 48 ms degraded trace	the centre holds no domain capability
2	Rewards discovered	m-rew-10, prompt 8ab1, A2A v1.0	capability is discovered, not shipped
3	Same question, new answer	routes to get_points_balance	capability grows with no central release
4	Transactions discovered	m-txn-10, same onboarding gate	the gate is a rule, not a favour
5	Cross domain journey	T1001, \$100, restaurant 3x, 300 points	the centre composes, domains stay decoupled
6	Prompt change detected	8ab1 to 91fd by snapshot diff	the platform detects change, not the author
7	Regression caught	case c13 expected 300, observed 200	targeted evidence catches a real defect
8	Repair certified	91fd to c3e4, 18 of 18 pass	the same loop certifies the repair
9	Tool added	get_expiring_points, schema t2e9	the registry expands without central edits
10	Skill added	expiring_points skill, m-rew-12	one domain ships new customer capability
11	Domain failure	Rewards span errors at 12 ms	failure is contained and named
12	Attested summary	reachability-closed, verdict ATTESTED	reachable surface equals published bundle

What the prototype does not prove

The demonstration is a deterministic replay of recorded events, not a load test. It establishes that the control surfaces exist and that the seams are in the right places. Every latency and throughput figure in Part IV is third-party or modelled, and must be re-measured against your own workload before it is used to buy hardware.

Figure 3. The twelve steps, the concrete artefact each one produces, and the claim it makes testable. The most important row is step 7: a semantic regression that is caught not by a general-purpose safety filter but by a domain evaluation case selected automatically from the shape of the change.

Steps 1 to 4 establish that capability is discovered rather than shipped. In step 1 the central runtime is running with no domain connected. A customer asks how many points they have, the planner runs, and the trace completes in 48 ms with a degraded status and the answer "No available domain capability can answer this request." This is the single most useful frame in the demonstration, because it shows the centre with its domain knowledge removed and therefore shows exactly how much domain knowledge the centre holds, which is none.

In step 2 the Rewards domain publishes a bundle. The centre discovers an agent card declaring version 1.0, prompt version 1.0 with fingerprint 8ab1, a protocol interface of A2A v1.0 / JSON-RPC, a bundle hash of m-rew-10, three skills, and two tools with schema hashes t4c1 and t8d2. Discovery does not confer trust. The platform selects an onboarding suite of intent and rewards smoke tests, runs five cases, and only then hands the result to a rule-based auditor which runs four checks and returns a verdict of ATTESTED. Step 3 replays the identical customer sentence from step 1 and it now routes through the planner to the Rewards agent to get_points_balance, returning "You have 12,500 points." No central code changed between steps 1 and 3. Step 4 repeats the whole gate for the Transactions domain and bundle m-txn-10, which matters because it shows the gate is a rule applied uniformly rather than a courtesy extended to the first domain.

Step 5 establishes that composition is a central responsibility. The cross-domain question about a Demo Bistro transaction produces a trace in which the planner fans out to Transactions, which returns transaction T1001 for \$100 in the restaurant category, and then to Rewards, which applies the 3x multiplier, yielding 300 points. The structural point is visible in the trace shape rather than in the answer: there is no span in which Transactions calls Rewards. The two domains remain unaware of each other, which is what allows either to be replaced without renegotiating with the other.

Steps 6 to 8 are the heart of the argument. In step 6 the Rewards team independently swaps its prompt. The platform does not receive a declaration of what changed; it compares runtime snapshots and observes the fingerprint move from `8ab1` to `91fd`, prompt version 1.0 to 1.1. From the change type alone it selects four suites: `intent`, `rewards_regression`, `cross_domain` and `response_judge`.

In step 7 those suites run. Seventeen of eighteen cases pass. Case `c13`, "Cross-domain Rewards Calculation", expected `300 points` and observed `200 points`, with the recorded reason "Bad prompt applied grocery 2x instead of restaurant 3x." The release status becomes `REGRESSION_DETECTED`, and the auditor withholds attestation because its `eval-executed` check requires an evaluation verdict of `PASS`.

It is worth dwelling on what kind of defect this is, because it is the defect class that general guardrails cannot see. The answer "Your Demo Bistro purchase earns 200 points" is well formed, on topic, non-toxic, free of injected instructions, grounded in a real transaction that was really retrieved, and confidently phrased. Every input rail and every generic output rail passes it. It is caught only because a domain-owned expectation about a domain-owned rule was selected for execution by a platform-owned policy triggered by a change the platform detected itself. Remove any one of those four properties and the wrong answer reaches the customer.

Step 8 closes the loop. The Rewards team ships a corrected prompt, the fingerprint moves `91fd` to `c3e4` and version 1.1 to 1.2, the same four suites re-run, all eighteen cases pass, and attestation is granted. No central change was involved in either the failure or the repair, which is the autonomy claim demonstrated rather than asserted.

Steps 9 and 10 establish that capability grows from the edge. Rewards registers a new tool, `get_expiring_points` with schema hash `t2e9`, and the platform selects tool-selection and rewards regression suites. Then Rewards adds an `expiring_points` skill. Step 10 shows the before and after explicitly: the question "When do my points expire?" is unsupported, then the skill lands, then the identical question routes through Rewards to `expiring_points` to `get_expiring_points` and returns a grounded answer. Both changes are confined to one domain's bundle, moving it to `m-rew-12`.

Step 11 establishes fault containment. Rewards is taken offline. A Transactions-only question succeeds normally, completing in 130 ms. The cross-domain question then produces a trace in which the Transactions work succeeds and the Rewards delegation span fails after 12 ms with an error status, and the trace as a whole is marked degraded rather than failed. The customer receives a named partial: the transaction was found, and Rewards is unavailable so points cannot be calculated right now. Two properties are on display. The failure is fast, because a per-dependency budget bounded it rather than a global timeout absorbing it. And the failure is legible, because the response says which capability is missing instead of silently omitting it, which is the difference between a degraded answer and a misleading one.

Step 12 states the attested end state. Rewards at version 1.2, prompt `c3e4`, bundle `m-rew-12`, with the reachable surface confirmed identical to the published bundle.

4.3 The auditor, and why it is not a model

The auditor runs four checks on every change, and their construction is deliberate.

Table 2. The four auditor checks in the reference implementation, with the comparison method each uses. Every one is a hash comparison, a set comparison, or a rule, and none involves a model.

Check id	What it establishes	Method	Failure meaning
<code>manifest-published</code>	A bundle with this hash was actually published and is the subject under audit	hash comparison	The artefact under review is not the artefact that was published
<code>reachability-closed</code>	The skills and tools central can reach are exactly the published set, with nothing missing and nothing extra	set comparison	Central can reach a surface that is not identical to the published bundle
<code>eval-coverage</code>	Every suite required for this change type was selected	rule	The evidence gathered does not cover the change that was made
<code>eval-executed</code>	The selected suites ran against this manifest and returned a passing verdict	rule	Evidence exists but does not support release

The verdict is `ATTESTED` only if all four pass, and `WITHHELD` otherwise.

Two aspects of this deserve emphasis because they are the parts most often argued with. First, the auditor is rule-based rather than model-based, which is a constraint accepted on purpose. A model-based auditor would be more flexible and would be able to comment on things a rule cannot, and it would also be non-deterministic, expensive to run on every change, and impossible to defend in a post-incident review, because the answer to "why did it pass" would itself be a generation. A rule-based verdict is reproducible, cheap enough to run on every change without negotiation, and settles arguments rather than starting them.

Second, `reachability-closed` is the check that makes the rest of the architecture load-bearing. It is a closed-world assertion: not merely that everything published is reachable, but that nothing else is. Without it, "we only evaluated the affected surface" is an assumption. With it, that sentence is a computed property, and targeted evaluation becomes a sound practice rather than an economising one.

4.4 What the prototype does not yet do

Stated plainly, so that no reader has to discover it themselves. There is no inference: the events are replayed, so no latency figure in the prototype reflects model behaviour. There are only two domains, so the coordination costs that appear at twenty are not exercised. There is no durable execution engine, so long-running and human-in-the-loop journeys are described in this document but not demonstrated. The guardrail cascade of Chapters 16 and 17 is designed here and not implemented there. And the prototype runs one model family throughout, `gpt-4.1-mini` in the recorded configuration, so the tiering strategy of Chapter 14 is likewise architecture rather than demonstration. Chapter 31 sequences the work to close these gaps in the order that retires the most risk first.

PART II

Why This Design, Adversarially

Part II is the argument. Each competing design is stated in its strongest form, credited with the cases where it genuinely wins, and then criticised by naming the specific mechanism through which it fails. Every criticism is paired with the observation that would refute it. Chapter 11 turns the same treatment on the design this document advocates.

5. How to Argue About Architecture

Architecture debates go badly for a predictable reason: the participants compare a well-understood incumbent against an idealised alternative, or an idealised incumbent against a caricatured alternative, and no observation is nominated in advance that could settle the disagreement. The chapters that follow adopt three rules to avoid that.

Steelman first. Each alternative is described as its best advocates would describe it, including the cases where it is simply the correct choice. An argument that only defeats the weak version of a position has established nothing.

Name the mechanism. "It does not scale" is not a criticism. A criticism identifies the specific quantity that grows, what it grows with, and what breaks when it does. Change surface growing linearly with capability count is a mechanism. Trust edges growing quadratically with peer count is a mechanism. Prefill tokens growing with catalogue size is a mechanism.

State the falsification. Each chapter ends by naming what would have to be true for the criticism to be wrong. This is the discipline that separates an architectural position from a preference, and it also makes the document useful in review: a reader who believes an alternative is being treated unfairly has a concrete experiment to propose.

5.1 The criteria

Six criteria are used consistently, chosen because each one is observable rather than aesthetic.

Table 3. The evaluation criteria used throughout Part II, and how each is observed rather than asserted.

Criterion	Question it answers	How it is observed
Change surface	When one team changes one thing, how many components must be re-verified?	Count the components a change-impact analysis returns, and check whether that count is computed or estimated
Blast radius	When one component fails or misbehaves, how many journeys degrade?	Fail one component in a controlled test and count affected journeys
Authority integrity	Can a model cause an action it was not entitled to cause?	Attempt it; if the answer depends on prompt wording, authority is not enforced
Attribution	For a given production request, can you reconstruct every decision and its cost?	Pick a request id at random and try
Deployability	Can one team ship without coordinating with another?	Measure elapsed time from merge to production for a single-domain change
Deadline ownership	Who is accountable for the end-to-end latency the user experiences?	Ask for a name; if there is not one, there is no budget

5.2 On premature decomposition

One concession belongs at the front, because it is the most common and most expensive way to misapply this document. Decomposing a system before its boundaries are understood is worse than not decomposing it. The boundaries in a federated agent platform are the domain agent cards, and a card drawn around the wrong capability set is expensive to move: it has consumers, versions, evaluation suites and an audit history. A team that does not yet know which capabilities belong together should build a single agent, keep it clean, and split it when the seams become obvious from the change log rather than from a diagram. Chapter 31 sequences the roadmap accordingly, starting with in-process modularity and moving to service federation only when a named pressure demands it.

Decision. Federation is not a starting position. It is a response to specific, observable pressures: more than three teams needing independent cadence, a regulatory requirement for reproducible authority decisions, a change surface that has already made full regression unaffordable, or a fault-containment requirement that shared-process modularity cannot meet. Adopt it when at least one of those is true and you can name which.

6. Alternative A: The Single Agent With Skills

6.1 The design in its strongest form

One agent process holds one system prompt. Every capability the organisation exposes appears in that prompt as a skill: a name, a description of when to use it, a parameter schema, and often a few examples. The agent receives the user's message, selects a skill, calls the corresponding tool, and answers. Modern frameworks make this genuinely elegant, and the operational story is the simplest of any option in this document: one deployable, one prompt file, one place to look when something is wrong, one trace, no network hops between reasoning and capability selection.

It also has a real latency advantage that is often overlooked in these comparisons. Skill selection happens inside the same model call that produces the plan, so there is no separate routing round trip, and there is no service boundary between the centre and the capability. On a request that a federated design would resolve in 460 ms, a well-tuned single agent with a small catalogue can plausibly reach 380 ms, because it has removed two network hops and one model call.

Where it is simply the right answer. A product owned by one team. A capability catalogue in the low single digits that is not expected to grow quickly. A prototype whose purpose is to discover what the boundaries are.

An internal tool where attribution requirements are informal. In these cases federation adds control planes that solve problems the system does not have, and this document's design would be the wrong recommendation.

6.2 Mechanism one: change surface grows with the catalogue, and it grows in a shared mutable global

The system prompt is a shared mutable global variable. That framing is not rhetorical; it has the same consequences that shared mutable state has anywhere else, and they are worth spelling out.

When the rewards team needs to correct its multiplier language, it edits a region of a file that also governs how transactions are looked up, how out-of-scope requests are refused, and how the agent decides between two similar capabilities. There is no mechanism that confines the effect of the edit to rewards, because the model attends over the whole prompt. The rewards team therefore cannot know, from the diff alone, what else it might have changed. Neither can anyone else. The only sound response is to re-verify everything, which is exactly what the reference implementation's federated design avoids: in the federated case a prompt change inside Rewards moved one bundle hash from `m-rew-11` to `m-rew-12` and triggered four named suites, and the auditor independently confirmed that nothing outside that bundle became reachable.

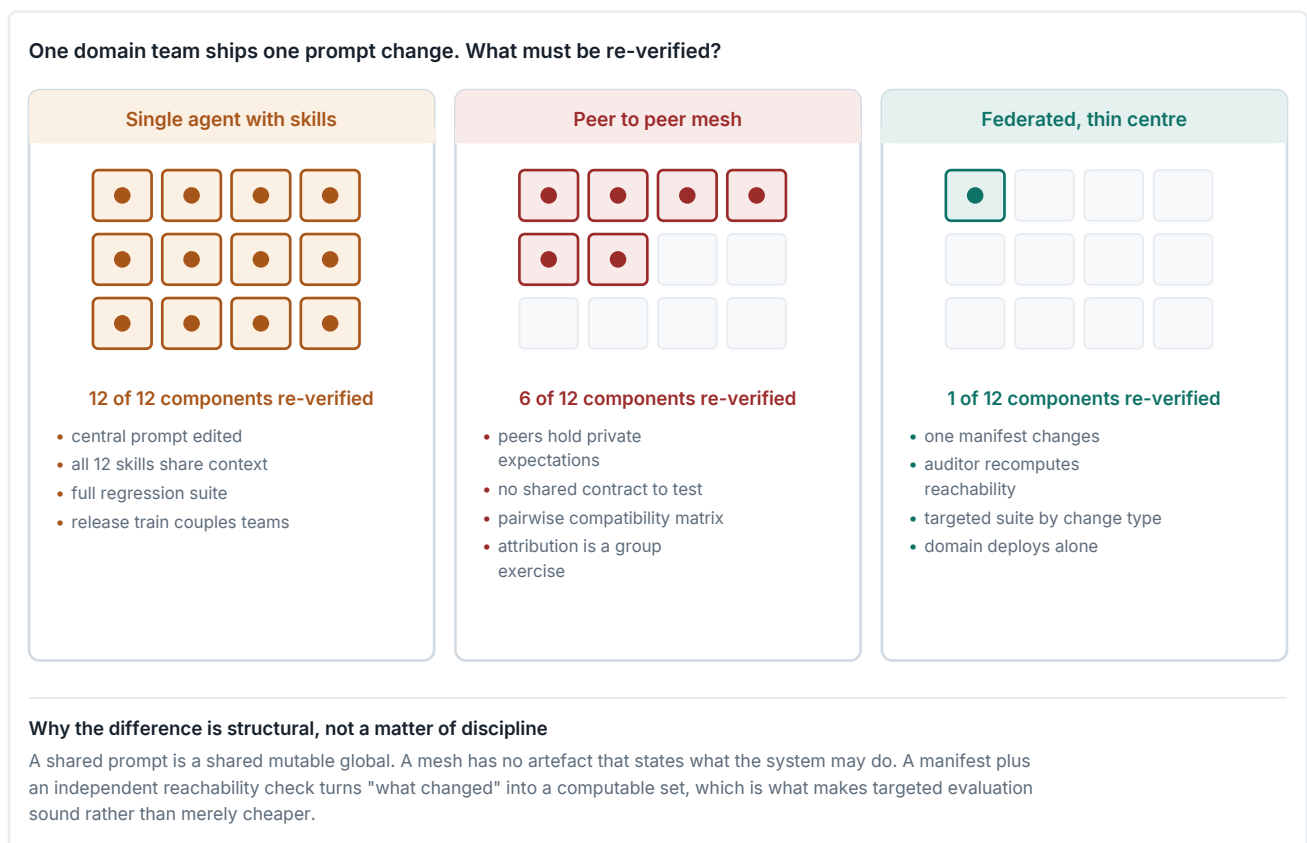


Figure 4. The same change under three architectures. The difference is not that federated teams are more careful; it is that a manifest plus an independent reachability check turns "what changed" into a computable set. Without that artefact, the honest engineering answer to "what must we re-test" is "everything", and the practical answer becomes "whatever we have time for", which is not the same thing.

The second-order consequence is worse than the first. Because full regression is expensive, teams reduce it. Because reductions are chosen by the person who made the change, they are chosen by the person least able to see the change's non-local effects, and with the strongest incentive to conclude the change is safe. The failure mode is not that a team is dishonest. It is that a reasonable engineer, asked to select the evidence that their own change is harmless, will select evidence consistent with that belief, and there is no artefact contradicting them.

6.3 Mechanism two: prefill cost grows with capability count, on every turn, whether or not the capabilities are relevant

Every capability description occupies context on every request. A moderately documented capability, with a description, a parameter schema and one example, occupies on the order of 150 to 250 tokens. Forty capabilities therefore add roughly 6,000 to 10,000 tokens of prompt that must be processed before the first output token can be produced, on every single turn, including the seventy percent of turns that concern exactly one capability.

The following is modelled, from the stated assumption of a single-stream prefill rate around 25,000 tokens per second for an 8B-class model on one H100 at FP8. It is offered as arithmetic to be redone with measured numbers, not as a benchmark.

Table 4. Modelled prefill contribution to time to first token as the capability catalogue grows, at 200 tokens per capability description and 25,000 tokens per second single-stream prefill. The final column is the fraction of a 500 ms conversational budget consumed before any reasoning has happened. Modelled, not measured.

Capabilities in prompt	Catalogue tokens	Modelled prefill contribution	Share of a 500 ms budget
4	800	32 ms	6%
12	2,400	96 ms	19%
40	8,000	320 ms	64%
120	24,000	960 ms	192%

Prefix caching mitigates this substantially and should always be used, but it mitigates the compute rather than the consequence. Two effects survive caching. The first is that the catalogue still occupies context window, competing with conversation history and retrieved evidence for the space that actually improves answers. The second is instruction dilution: as the number of simultaneously active instructions grows, adherence to any individual instruction degrades, and the degradation is not uniform. It falls hardest on the instructions that discriminate between similar capabilities, which is precisely the routing decision the prompt exists to make. A federated design confronts the model with a shortlist of two or three eligible capabilities produced by a cheap deterministic or Tier 1 step, so the discrimination problem the model faces is small regardless of how large the organisation's catalogue becomes.

6.4 Mechanism three: prompt shadowing, the failure that domain evaluation cannot see

This mechanism deserves its own treatment because it is specific, it is common, and it is nearly invisible to the obvious tests.

Consider the reference implementation's rewards rule. Suppose the central prompt contains, from some earlier iteration, the sentence "point multipliers are 2x for everyday spending categories". Suppose the rewards team then discovers its multiplier bug and corrects its own guidance to state the restaurant 3x rule explicitly. In a federated architecture the corrected guidance lives inside the Rewards bundle, is the only rewards guidance in play, and the corrected behaviour is confirmed by case c13 expecting 300 points.

In a single-agent architecture both statements are in the same prompt. Which one wins is a function of ordering, phrasing, recency and the model version, none of which the rewards team controls or can test against, because the rewards team does not own the file. Worse, the rewards team's own evaluation may pass, because the team will naturally test the capability in isolation with a prompt fragment that omits the contradicting sentence. The bug surfaces in production, intermittently, and diagnosis requires someone to notice a contradiction between two paragraphs written months apart by different people. This is a well-known category of defect in shared configuration, and language models make it harder to detect rather than easier, because the system does not fail loudly on contradictory instructions; it silently picks one.

6.5 Mechanism four: every domain release becomes a system release

Because there is one deployable and one prompt, there is one release. A rewards correction and a transactions migration that happen to land in the same week are now coupled: they are tested together, released together, and rolled back together. The rewards team's urgent fix waits for the transactions team's risky migration to be ready, or the migration is rushed. Neither outcome is a technology problem, and neither is solvable by better tooling within this architecture, because the coupling is structural.

The observable consequence is a metric worth tracking in any organisation currently on this path: the elapsed time from a domain team's merge to that change being live, and specifically the fraction of that time spent waiting for other teams. When that fraction passes about a third, the architecture is imposing a tax that federation exists to remove.

6.6 What would falsify this criticism

The criticism of the single-agent pattern is wrong if all of the following can be demonstrated. First, that routing accuracy over a catalogue of forty or more capabilities stays within about one point of the accuracy achieved over four, measured on a held-out set that includes near-miss pairs. Second, that a domain team can edit its own region of the prompt and merge without review by the prompt's owning team, with the effect of that edit confined provably to its own capabilities. Third, that the set of components requiring re-verification after a prompt edit can be computed from the diff rather than estimated. And fourth, that per-domain circuit breaking, quota and cost attribution can be implemented without a service boundary.

If those four hold, the single-agent design dominates on latency and operational simplicity and this document is recommending unnecessary machinery. In practice the first fails as catalogues grow, and the second fails for the reason described in Chapter 2: ownership is real, and a shared file has one owner.

7. Alternative B: Peer to Peer, Without a Centre

7.1 The design in its strongest form

Domain agents talk to each other directly. Rewards, needing a transaction, asks Transactions. Transactions, needing an entitlement decision, asks Policy. There is no orchestrator, no single point of failure, and no bottleneck team through which every new capability must pass. The topology is attractive on paper for real reasons: it removes the central team from the critical path of both requests and organisational change, it degrades gracefully in the sense that no single node's loss stops all communication, and it is a good model of how human organisations actually coordinate.

Where it is genuinely the right answer. Two systems with a stable, narrow, well-specified contract that has no reason to involve a third party. Cross-organisational settings where no participant can legitimately be appointed as the centre, which is the case protocols for agent interoperability are designed for. And workloads where quality is a statistical property of many attempts rather than a per-request guarantee, such as exploratory research pipelines, where emergent coordination is a feature.

7.2 Mechanism one: nobody owns the end-to-end deadline

This is the criticism that matters most, and it is not about the mean.

Each peer, reasonably, sets its own timeout for its own dependencies. No peer knows the user's remaining budget, because the budget was established at the edge and there is no mechanism carrying it. A chain of four or five delegations therefore has a latency distribution that is the convolution of independently chosen distributions, and the tail behaves much worse than intuition suggests. If each hop has a p99 of 400 ms, the chain's p99 is not 400 ms and it is not simply the sum either; it is dominated by the probability that at least one hop is slow, which rises with the number of hops. With four hops, the chance that none of them takes its p99

excursion is about 96 percent, so roughly one request in twenty-five experiences at least one worst-case hop, and that request has no mechanism to abandon the slow hop because no participant knows the deadline has already been exceeded.

The fix is a deadline propagated with the request and enforced at each hop against the remaining budget. That fix is exactly a central contract about how time is spent, and it requires someone accountable for setting the budget in the first place. The moment it exists, a centre exists.

7.3 Mechanism two: no closed-world reachability, therefore no computable blast radius

In the reference implementation, the auditor's `reachability-closed` check asserts by set comparison that the skills and tools the platform can reach are exactly the published set. That assertion is possible because there is one place through which delegation flows and one registry describing what may be delegated to.

In a peer topology, no participant can make that assertion, because reachability is a transitive property of a graph that no single node observes. If Rewards may call Transactions, and Transactions may call Policy, then Rewards can reach Policy, whether or not anyone intended it and whether or not Rewards is entitled to. Answering "what is this system permitted to do" requires collecting every peer's configuration and computing a transitive closure, which means it can only be answered as an offline audit exercise, which means it is stale by the time it is finished, which means it will not be done on every change. Consequently every claim of the form "we tested the affected surface" is unverifiable, and targeted evaluation stops being a sound practice and becomes an economy.

7.4 Mechanism three: trust and compatibility edges grow quadratically

Each pair of communicating peers requires a contract, a version compatibility relationship, an authentication and authorisation relationship, an agreed error taxonomy, and a pair of integration tests. The number of such relationships is the number of edges in the communication graph.

Table 5. Relationship count as peer count grows, assuming a fully connected graph in the worst case and a realistic sparse graph at three connections per peer. The right-hand column is what actually breaks: each edge is an artefact somebody must own, version and test. Modelled from graph arithmetic.

Peers	Edges if fully connected	Edges at 3 connections per peer	Federated: hub relationships
5	10	8	5
10	45	15	10
20	190	30	20
40	780	60	40

The honest version of this criticism acknowledges that real peer topologies are sparse, not complete, so the quadratic bound is a worst case rather than a forecast. But the middle column tells a story of its own: even sparsely connected, the number of pairwise artefacts exceeds the number of hub relationships and, more importantly, the relationships are heterogeneous. Twenty hub relationships are twenty instances of one contract shape, which can be tooled, validated and versioned uniformly. Thirty peer relationships are thirty bilateral negotiations, each with its own history.

7.5 Mechanism four: delegation loops are emergent rather than designed

A calls B, B needs something and calls A. In a small deployment this is caught in review. In a deployment where each team independently decides its own dependencies, it appears as a production incident: a request that consumes resources on several nodes, produces no answer, and cannot be attributed to any single team's change. Preventing it requires a hop budget carried with the request, cycle detection over a graph somebody maintains, or a rule that delegation may only flow in one direction. All three are central coordination mechanisms.

7.6 Mechanism five: no trace root, so attribution becomes archaeology

The reference implementation's step 11 trace shows the property this mechanism destroys. One trace, one root, a Transactions delegation succeeding, a Rewards delegation failing after 12 ms, an overall status of degraded. A reviewer can see in one artefact what was attempted, what failed, how long it took and what the customer was told.

Distributed tracing across peers is achievable, and mature organisations do it. But it is achievable precisely to the extent that all peers agree on a propagation format, a span naming convention, a sampling policy and a collector. That agreement is a central standard, enforced centrally, or it does not hold; and partial adoption is close to worthless, because a broken chain in the middle of a trace hides exactly the hop that is usually at fault.

7.7 What would falsify this criticism

The criticism is wrong if a peer topology can produce, on demand and without an offline exercise: an artefact stating the closed set of actions the system may currently take; a named owner of the end-to-end deadline with a mechanism that enforces it at each hop; and a single trace, rooted at the user request, spanning five delegations with per-hop cost and status. If all three exist, the criticism fails on its own terms. It is also worth noting the likely consequence of building them: a registry, a deadline contract and a trace standard together constitute a control plane, and a peer topology with a control plane is the design this document advocates, described differently. At that point the disagreement is terminological rather than architectural, which is a good outcome.

8. Alternative C: The Monolith Repository

8.1 The design in its strongest form

All domains live in one repository and deploy as one artefact. There is no service discovery, no network partition between components, no version skew, and no distributed transaction. Refactoring across domain boundaries is a single atomic commit. Testing is straightforward because the whole system can be instantiated in one process. A junior engineer can read the entire request path in an afternoon.

These are large advantages and this document does not dismiss them. Most organisations that decomposed early regret it, and the literature on premature decomposition is more persuasive than the literature encouraging it. If the choice is between a well-maintained monolith and a distributed system nobody can debug, the monolith is the better engineering outcome.

Where it is genuinely the right answer. Fewer than about three teams. A domain model still in flux, where the correct boundaries are not yet known. A latency budget so tight that inter-service hops cannot be afforded. And any organisation without the operational maturity to run a distributed system, because a distributed system operated badly is strictly worse than a monolith operated well.

8.2 Mechanism one: release coupling compounds with team count

If a shared release requires every participating team's changes to be ready and passing, then the probability that a given release proceeds on schedule is the product of each team's readiness probability. Even at a generous ninety-five percent readiness per team, six teams yield about a seventy-four percent chance of an unblocked release, and twelve teams about fifty-four percent.

Table 6. Modelled probability that a shared release is unblocked, as team count grows, at two per-team readiness rates. The consequence is not the arithmetic itself but the behaviour it induces: teams begin batching changes to reduce the number of releases they must pass through, which makes each release larger and riskier. Modelled from independence assumptions, which are optimistic.

Teams sharing the release	At 95% per-team readiness	At 90% per-team readiness
3	86%	73%
6	74%	53%
12	54%	28%
20	36%	12%

The assumption of independence is optimistic, because teams blocked by the same shared dependency fail together. The induced behaviour is the real cost: as the probability of a clean release falls, teams batch more changes into each attempt, which raises the risk of each attempt, which increases the change-control burden, which lengthens the cycle further. Organisations in this state usually describe the problem as insufficient test automation. More automation helps, and it does not address the coupling.

8.3 Mechanism two: continuous integration wall clock grows with total capability

A monolith's test suite covers the whole system, so its wall-clock duration grows with the total capability of the organisation rather than with the size of any change. Parallelisation buys a constant factor and then runs into shared fixtures, database migrations and integration tests that cannot be parallelised safely. Eventually the suite takes longer than the desired release interval, and at that point one of three things happens: the interval lengthens, the suite is subsetting by hand, or the suite is subsetting by a heuristic nobody has validated. The second and third are the common outcomes, and both quietly abandon the guarantee that made the monolith attractive.

The federated alternative does not eliminate this cost; it partitions it. Each domain's suite grows with that domain, and the platform's contract suite grows with the number of contracts rather than with the behaviour behind them.

8.4 Mechanism three: one dependency vulnerability forces a full-platform redeploy

A critical vulnerability in a transitive dependency requires the whole artefact to be rebuilt, re-tested and re-released, including domains that do not use the affected code path. Under a remediation deadline this pulls every team into an unplanned release, and the risk of that release is the risk of everything currently on trunk, not the risk of the patch. Independently deployed domains upgrade on their own schedule, and the blast radius of the emergency is the domains that actually use the dependency.

8.5 Mechanism four: no independent scaling, and no independent failure

One process means one resource envelope. A retrieval-heavy domain that wants large memory forces that memory allocation on every replica, including the replicas serving requests that never touch it. More seriously, the failure modes are shared: a thread pool exhausted by one domain's slow dependency starves every domain in the same process, a memory leak in one module degrades all of them, and a garbage collection pause caused by one domain's allocation pattern is experienced by every request in flight. There is no per-domain circuit breaker to open, because there is no boundary at which to open one.

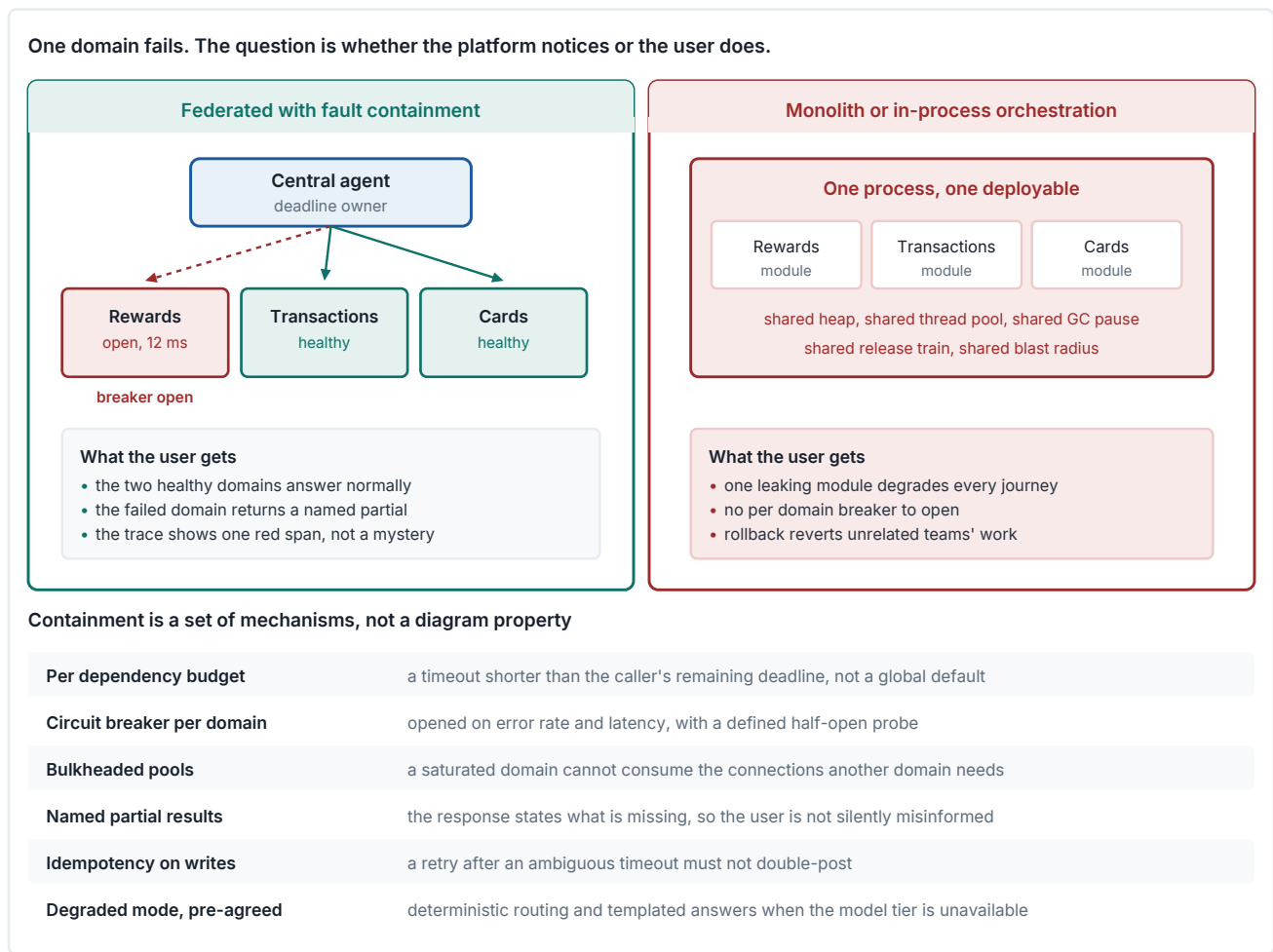


Figure 5. The same fault under two architectures, and the mechanisms that make the difference. Note that fault containment is not a property of drawing boxes with gaps between them. It is the presence of six specific mechanisms: per-dependency budgets, per-domain breakers, bulkheaded pools, named partial results, idempotent writes, and a pre-agreed degraded mode.

8.6 What would falsify this criticism

The criticism is wrong if a monolithic repository can demonstrate that a single team's change reaches production within an hour of merge without any other team's involvement; that the test subset required for a given change is computed from the change rather than chosen by a person; that a memory leak or thread pool exhaustion in one module can be contained without restarting the others; and that one domain's resource requirements can be satisfied without imposing them on all replicas.

Modular monoliths with strict module boundaries, per-module test selection, trunk-based development and feature flags get closer to this than critics usually allow, and an organisation that achieves it has captured most of federation's benefits at much lower operational cost. It is worth noticing what such a system has become, though: independently releasable units with enforced boundaries and computed change impact. The remaining difference is process isolation, which is the subject of Chapter 10.

9. Alternative D: Federated, But Ungoverned

9.1 The design in its strongest form

This is the nearest neighbour to the design this document advocates, and therefore the most important alternative to treat carefully. Domains are independently owned and deployed. A central runtime routes. Each domain publishes a card describing its capabilities. Each domain owns its own evaluation suites, and runs them before release. There is no independent auditor, because each team is trusted to test its own work, which is the normal and correct default in software engineering.

The steelman is strong. Teams are competent and accountable. Adding an auditor implies distrust, creates a bottleneck, and duplicates work the domain team is better placed to do. Most successful microservice organisations operate exactly this way, and they do so successfully.

9.2 Mechanism one: evidence selected by the changing team is not independent evidence

The argument here is not about trust; it is about the structure of the task.

To select the right evaluation suites for a change, someone must know the change's non-local effects. The team making the change knows the change best and its non-local effects least, because non-local effects are by definition outside the team's area of ownership. So the team best placed to select evidence is, for this specific question, poorly placed. Add the ordinary incentive that suite selection happens under release pressure, and the result is not misconduct but a systematic bias: the selected evidence is consistent with the belief that the change is safe.

The reference implementation makes the alternative concrete. The platform detected the change itself, by comparing runtime snapshots and observing the prompt fingerprint move from `8ab1` to `91fd`. It selected the suites from the change type, by rule. Case `c13`, the cross-domain calculation, was in that selection because a prompt change in a domain participating in cross-domain journeys implicates those journeys. It is entirely plausible that a rewards engineer, having verified that balance lookups still work and the multiplier language reads correctly, would not have chosen a cross-domain composition test. That case is precisely the one that failed.

9.3 Mechanism two: declared capability drifts from actual capability

Without a check that the reachable surface equals the published surface, the two diverge, and they diverge in the direction that matters. A tool is added for a debugging purpose and remains callable. A skill is deprecated in the card and remains wired. A parameter schema is widened in code and not in the card. Each divergence is individually harmless and locally rational. Collectively they mean the card is documentation rather than a contract, and every downstream property that depends on it, including the entire targeted evaluation argument, is unfounded.

The check that prevents this is cheap. `reachability-closed` is a set comparison between the published bundle and what the runtime can reach. It requires no model, no human judgement and no negotiation, and it runs in milliseconds. The reason to make it independent of the changing team is not suspicion, it is that a check a team runs on itself is a check that can be disabled under pressure by the same team, whereas a check in the release gate cannot.

9.4 Mechanism three: nothing establishes that evidence was gathered against the artefact being released

The `eval-executed` and `manifest-published` checks exist to close a gap that is easy to miss. A team can run a full evaluation suite, pass it, then make one more small change, and release. The evidence is real, the verdict is genuine, and neither pertains to the artefact that shipped. Binding the verdict to a manifest hash makes the gap closed by construction rather than by process discipline.

9.5 What would falsify this criticism

The criticism is wrong if an ungoverned federation can produce, for an arbitrary past release chosen by someone else: the exact set of reachable capabilities at that moment; the evidence gathered, bound to that artefact's hash; and the rule by which that evidence was selected. If those three artefacts exist and were produced without the changing team's participation, then governance is present and the label is the only thing in dispute.

The strongest genuine objection to this chapter is that the auditor becomes a bottleneck. That objection is answered by construction rather than by argument: because every check is a hash comparison, a set comparison or a rule, the auditor is a function in the release pipeline that completes in milliseconds and requires no human. It is a bottleneck in the same sense that a type checker is a bottleneck.

10. Alternative E: In-Process Orchestration Versus Service Federation

10.1 The design in its strongest form

Modern agent frameworks support multi-agent composition inside a single process. A supervisor agent delegates to sub-agents; each sub-agent has its own prompt, its own tools, and its own evaluation suite. The code is genuinely modular, ownership can be expressed through code ownership rules, and none of the modularity is imaginary.

The advantages over service federation are real and should not be minimised. Delegation is a function call rather than a network round trip, saving perhaps 2 to 15 ms per hop plus serialisation, which is material inside a 460 ms budget. There is no service mesh, no discovery infrastructure, no distributed tracing requirement, and no partial failure between the supervisor and the sub-agent. Development is faster because the whole system runs locally. For a small number of domains this is not merely acceptable, it is the correct choice, and Chapter 31's roadmap begins here deliberately.

10.2 Mechanism one: modularity without fault isolation

The distinction that matters is between logical and physical boundaries. In-process composition gives a logical boundary: clear ownership of code and prompts, separate evaluation, independent reasoning about behaviour. It gives no physical boundary, and every failure mode that crosses a physical boundary is therefore shared.

One sub-agent whose retrieval step allocates aggressively causes garbage collection pauses experienced by every request in the process. One sub-agent that exhausts a shared connection or thread pool starves the others. One unhandled fault that terminates the process terminates all domains simultaneously. The reference implementation's step 11 depends entirely on a physical boundary: Rewards failed in 12 ms while Transactions continued to serve in 130 ms, because Rewards was a separate address space with its own budget and its own breaker. In one process, "Rewards is unavailable" is not a state the system can be in.

10.3 Mechanism two: one dependency graph, one runtime, one version

All sub-agents share a single dependency resolution. Two domains that need different major versions of the same library cannot both be satisfied. One domain's need for a newer runtime forces the upgrade on all of them. Over a few years this produces the familiar outcome where the whole platform is pinned to the oldest constraint any single domain imposes, and the team that needs to move cannot.

10.4 Mechanism three: no independent deployment, so no independent cadence

Because there is one process, there is one deployment, which recreates Chapter 8's release coupling at a smaller scale. The mitigations are the same, feature flags and trunk-based development, and so are their limits: a flag controls whether new code runs, not whether new code is present, and shipping a domain's change still means shipping every other domain's current state.

10.5 Mechanism four: no per-domain resource governance

A shared process cannot easily enforce that one domain uses at most a given share of CPU, memory, model quota or connections. The result is that a single misbehaving domain's blast radius is the whole platform, which is the property federation exists to prevent.

10.6 What would falsify this criticism

The criticism is wrong if an in-process design can demonstrate that one sub-agent can be restarted or disabled without affecting the others; that one sub-agent's memory and CPU consumption can be bounded; that two sub-agents can depend on incompatible library versions simultaneously; and that one team can deploy without shipping another team's current trunk state. Mechanisms exist that approach some of these, including separate class loaders, per-task resource accounting and in-process sandboxes. They approach process isolation by reimplementing it, usually with fewer guarantees and less mature tooling than the operating system provides.

11. Where This Design Is Worse

Everything above criticises alternatives. This chapter applies the same standard to the design this document advocates, without mitigation and without a rebuttal paragraph after each item. A reader who finds this chapter more persuasive than Chapters 6 to 10 should not adopt the architecture, and that is a legitimate outcome of a design review.

11.1 It adds latency to every request, forever

Two to four network hops are added to each journey. At 2 to 15 ms per hop including serialisation, that is roughly 5 to 60 ms of additional p50 latency, and materially more at p99 where connection pool exhaustion, retries and cold TLS handshakes appear. This cost is paid on every request, including the large majority that would have been resolved identically by a single in-process agent. It never amortises and it cannot be optimised away, only reduced.

11.2 Debugging is genuinely harder, and the tooling debt comes first

A single-process failure is a stack trace. A federated failure is a distributed trace, and only if tracing was correctly instrumented in every service before the incident. When it was not, engineers correlate logs across services by timestamp, which is slow and unreliable. The investment in tracing, correlation identifiers, structured logging and span conventions must be made before it pays for itself, which means it must be made at exactly the moment the team is least able to justify it. Organisations that skip it end up with a distributed system and monolithic debugging practices, which is the worst combination available.

11.3 There is a platform team headcount floor, and it does not amortise at small scale

Someone must own the central runtime, the registry, the auditor, the guardrail cascade, the trace collector, the evaluation runner and the model plane. That is realistically four to eight engineers with a specific and not-easily-hired skill mix, described in Chapter 25. Below roughly five domains, those engineers are a larger investment than the coordination cost they remove, and the honest recommendation at that scale is a modular monolith with a good test suite.

11.4 The control plane is a new correlated failure mode

Federation trades many small failure domains for a few shared ones. If the capability registry is unavailable, routing fails for every domain simultaneously, which is a worse failure than any single domain outage the architecture was designed to contain. The mitigations are known: aggressive caching of the registry with a last-known-good fallback, static routes for critical journeys, and a degraded mode that does not consult discovery at all. Each mitigation is additional machinery that must itself be built, tested and exercised, and an untested fallback path is not a fallback.

11.5 Contract and versioning overhead is a real tax on small changes

Adding one field to one response is, in a monolith, a one-line change. In this architecture it is potentially a schema version, a manifest hash change, a compatibility decision, a card update and a contract test. That overhead is proportionally trivial for a large change and disproportionately heavy for a small one, and it creates a standing incentive to batch small improvements, which is the behaviour the architecture is supposed to discourage.

11.6 Evaluation cost is partitioned but not reduced, and coordination cost is added

Each domain runs its own suites, and the platform runs contract, routing and cross-domain suites on top. Total evaluation compute is often higher than a monolith's, not lower; what improves is that the cost of any single change is bounded and attributable. Cross-domain journeys are also harder to evaluate, because doing it properly requires either a shared test environment where several domains are simultaneously at known versions, or contract-level mocking that risks passing tests while production composition is broken.

11.7 Cognitive load rises for everyone, including people who did not ask for it

A new engineer joining a domain team must understand the agent card format, the manifest and hash scheme, the registry, the delegation protocol, the trace conventions and the evaluation gate before shipping their first change. In a monolith they read the code. This cost is paid on every onboarding and it is a real drag on the pace of small teams.

11.8 Summary of when not to adopt this design

Table 7. Conditions under which this document's architecture is the wrong choice, with the recommendation that is better. Presented without qualification, because a design document that cannot say when it does not apply is marketing.

Condition	Better choice	Reason
Fewer than three teams contributing capability	Modular monolith, or a single agent with skills	Platform team cost exceeds the coordination cost removed
Domain boundaries not yet stable	Single agent, split later	Agent cards are expensive to move once they have consumers and history
No requirement for reproducible authority decisions	Simpler architecture, keep the guardrails	Most of the control plane exists to serve attribution requirements
Latency budget under about 300 ms end to end	In-process composition	The hop budget alone can consume a fifth of the allowance
No operational capability for distributed systems	Monolith, invest in operations first	A distributed system run badly is worse than a monolith run well
Exploratory or research workload	Peer to peer, or a single agent	Emergent coordination is an asset when per-request guarantees are not required

12. The Comparison, Quantified

The table below summarises Part II. It is deliberately blunt, and every column has been defined observably in Chapter 5 so that a reader who disagrees with a cell can say what measurement would change it.

Table 8. Comparison across the six criteria of Chapter 5, at a scale of roughly twenty domains and six contributing teams. Cells marked as modelled derive from the arithmetic in the chapters above. This table would look materially different at three domains, where the right-hand column's advantages largely disappear.

Criterion	Single agent with skills	Peer to peer, no centre	Monolith repository	In-process orchestration	Federated with a thin centre
Change surface for one domain edit	Whole prompt, all capabilities	Unbounded, not computable	Whole artefact	Whole process	One bundle, computed by rule
Is blast radius computable?	No, estimated by the author	No, requires transitive closure	No, estimated	No	Yes, from the manifest plus reachability check
Blast radius of one component failing	All journeys	Unpredictable, depends on graph	All journeys	All journeys	Journeys using that domain only
Authority integrity	Prompt dependent	Duplicated per peer, divergent	Enforceable, if code is disciplined	Enforceable, if code is disciplined	Enforced at one deterministic chokepoint
Attribution for one request	One trace, low resolution	Requires archaeology	One trace	One trace	One trace, per hop cost and status
Independent deployability	No	Yes	No	No	Yes
Deadline ownership	Implicit, single process	Nobody	Implicit, single process	Implicit	Explicit, the platform team
Serial hops on the critical path	1	3 to 6, unbounded	1	1	2 to 3, bounded by policy
Added p50 network latency	0 ms	20 to 90 ms, modelled	0 ms	0 ms	5 to 60 ms, modelled
Relationships to maintain at 20 units	1 shared prompt	30 to 190 edges, modelled	1 artefact	1 process	20 hub contracts
Probability a shared release is unblocked at 6 teams	74%, modelled	Not applicable	74%, modelled	74%, modelled	Not applicable, releases are independent
Platform team floor	None	None	None	Small	4 to 8 engineers
Onboarding cost for a domain engineer	Low	Medium	Low	Low	High
Where it is the right answer	1 team, small catalogue	Cross-organisation, research	Under 3 teams, unstable boundaries	Under 5 domains, tight latency	Many teams, attribution required

Read across the bottom two rows before reading the rest. The federated design wins on the criteria that dominate long-run cost in a large organisation, and it loses on the criteria that dominate short-run velocity in a small one. That is the trade, stated as plainly as it can be, and any presentation of this architecture that does not concede the last two rows is not making an argument.

PART III

Runtime Design

Part III is the implementable core: what the central agent does stage by stage and how its latency budget is divided, how model capability is tiered so that most turns never reach a large model, what happens to the same journey when the control path is moved to a frontier API, and how input and output guardrails are placed, layered and evaluated.

13. Anatomy of the Central Agent

13.1 Five responsibilities, and the refusal of a sixth

The central agent is the only component every request passes through, which makes it the most dangerous place in the architecture to put anything. Every responsibility it accumulates becomes a responsibility that cannot be changed without a platform release, cannot be owned by a domain team, and cannot fail without failing everything. The discipline is therefore negative as much as positive: the centre does five things, and the list of things it must never do is treated as part of the design rather than as an omission.

The five are classification, discovery, planning, authority enforcement and composition. Classification decides what kind of request this is and how much risk it carries. Discovery decides which declared capabilities are eligible, using the registry rather than any knowledge held in a prompt. Planning decides the smallest sufficient sequence of domain calls. Authority enforcement decides, deterministically, whether the identity behind the request may cause the effects the plan implies. Composition turns domain results into a response that has passed output validation.

Notice what these have in common. Each is a decision about the shape of the interaction rather than about the content of any domain. None of them requires knowing what a rewards multiplier is, and the reference implementation demonstrates this by removing every domain from the platform and observing that the centre degrades to a truthful refusal in 48 ms rather than answering badly.

Design rule. If a change to the central agent would be required in order to add, correct or remove a business rule, the boundary has been drawn in the wrong place. The centre may learn that a capability exists; it may never learn what the capability means.

13.2 Stage by stage, with a budget

A budget that is not divided is not a budget. The following division is the one this document defends, and it is stated as a target rather than a measurement so that a reader can dispute individual lines.

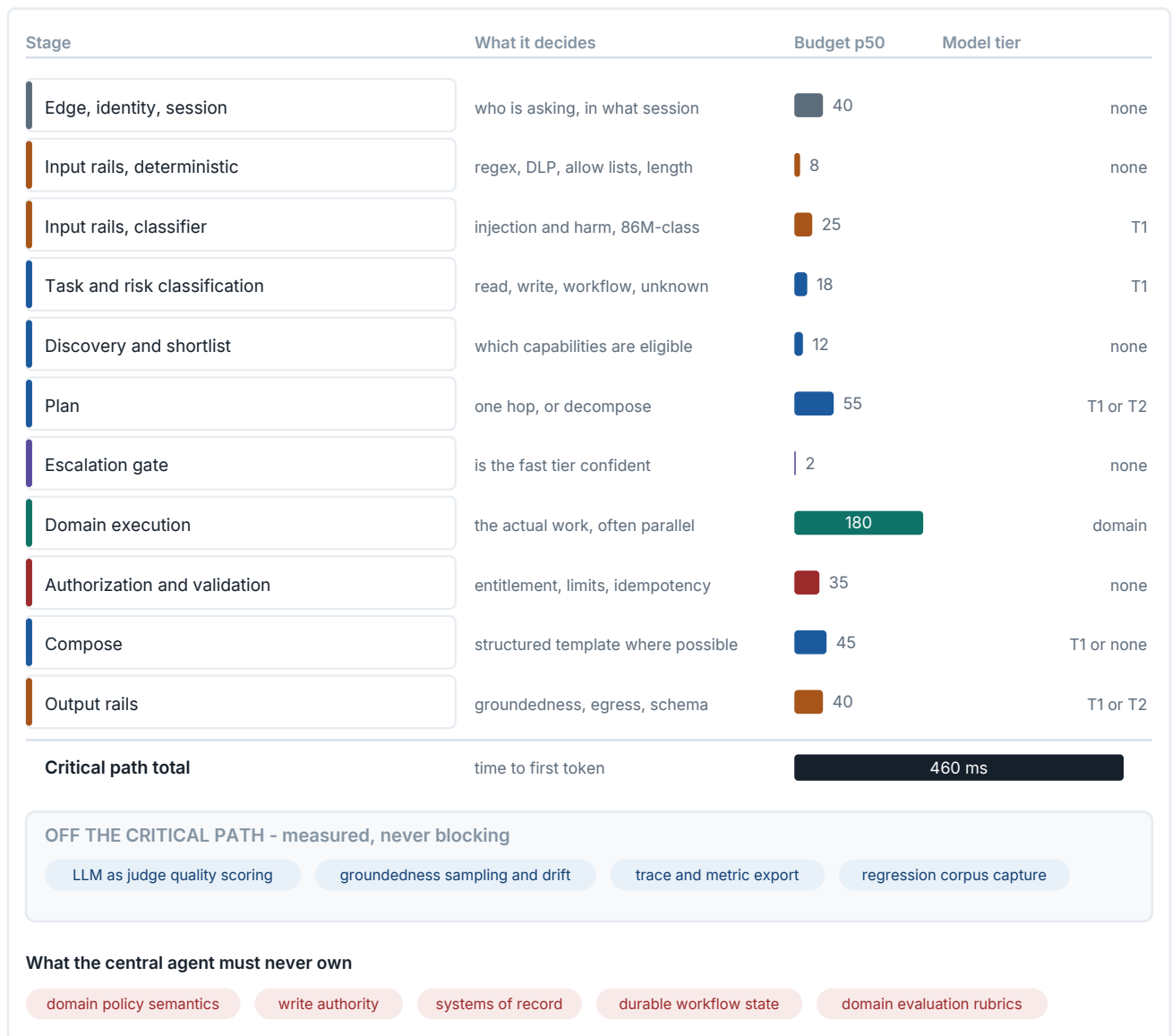


Figure 6. The central agent decomposed into stages, with the p50 budget each stage is allowed and the model tier it uses. Two features of the diagram carry most of the argument: the critical path contains at most two model calls before domain execution, and everything in the lower band is assurance work that is measured continuously and never allowed to block a user.

Table 9. The stage budget for a read-path journey, expressed as p50 targets summing to 460 ms, with p99 allowances stated per stage. The p99 column is a per-stage allowance, not a decomposition of end-to-end p99; the arithmetic of that distinction is discussed below. Modelled target budget, to be replaced by measurement.

#	Stage	What it decides	Model tier	p50 budget	p99 allowance
1	Edge, identity, session	Who is asking, in what session	none	40 ms	95 ms
2	Input rails, deterministic	Regex, DLP, allow lists, length	none	8 ms	14 ms
3	Input rails, classifier	Injection and harm, 86M-class	Tier 1	25 ms	70 ms
4	Task and risk classification	Read, write, workflow, unknown	Tier 1	18 ms	55 ms
5	Discovery and shortlist	Which capabilities are eligible	none	12 ms	40 ms
6	Plan	One hop, or decompose	Tier 1 or 2	55 ms	220 ms
7	Escalation gate	Is the fast tier confident	none	2 ms	3 ms
8	Domain execution	The actual work, often parallel	domain	180 ms	900 ms
9	Authorization and validation	Entitlement, limits, idempotency	none	35 ms	90 ms
10	Compose	Structured template where possible	Tier 1 or none	45 ms	160 ms
11	Output rails	Groundedness, egress, schema	Tier 1 or 2	40 ms	130 ms
	Total	Time to first token		460 ms	see below

Three observations about this table matter more than the individual figures.

The centre's own model usage is small and deliberately capped. Stages 3, 4, 6, 10 and 11 may involve a model, but stages 3, 4, 10 and 11 are Tier 1 work of tens of milliseconds, and stage 6 is the only stage permitted to reach Tier 2 on an ordinary turn. The sum of central model time on a typical read is well under 150 ms. This is what makes a conversational budget achievable at all, and it is the direct consequence of the tiering discipline in Chapter 14.

Domain execution is the largest single line and the platform team does not own it. At 180 ms it is 39 percent of the budget, and it is spent inside systems the platform team cannot optimise. The only levers the centre holds are parallel fan-out where the plan permits it, per-dependency budgets that bound the damage when a domain is slow, and a named partial result when a budget is exceeded. This is why Section 11.1 concedes network hops as a permanent cost: the hops are the price of the boundary that makes the 180 ms someone else's accountable number rather than an unowned one.

Adding the p99 column is not the same as computing the p99. The naive sum of the p99 allowances is 1,777 ms, and quoting that as the end-to-end p99 would be wrong, because it assumes every stage takes its worst case on the same request. The correct statement is weaker and more useful: the sum is an upper bound, the realistic end-to-end p99 is dominated by whichever single stage has the heaviest tail, and on this budget that is domain execution. The practical consequence is that tail-latency work should start at stage 8 and not at the stages the platform team finds most interesting to optimise.

13.3 What the centre must never own

The list below is the operative half of the design. Each row states a responsibility, why it is tempting to centralise it, and the specific failure that follows.

Table 10. Responsibilities excluded from the central agent by design, with the failure that follows from accepting each one. Every row has been accepted by some real system, which is why the list is worth writing down.

Excluded responsibility	Why it gets centralised anyway	What breaks when it does
Domain policy semantics	It is faster to add one sentence to the central prompt than to wait for a domain release	The centre becomes the authority on rules it cannot evaluate, and prompt shadowing follows, as in Section 6.4
Write authority	The model already knows what the user asked for, so letting it decide feels efficient	Authority becomes prompt dependent and cannot be reproduced for an auditor
Systems of record	A cache in the centre is quicker than a domain call	Two sources of truth, and the centre's copy is the one nobody owns
Durable workflow state	Holding the state in the session is simpler than adding an execution engine	A restart loses in-flight journeys, and long-running work cannot survive deployment
Domain evaluation rubrics	Central ownership makes the suites uniform	The team that can adjudicate correctness stops owning the definition of correct
Retry policy for domain internals	The centre sees the failure first	Retry storms amplify a domain incident, and idempotency assumptions leak across the boundary

13.4 Statelessness, sessions and the durable boundary

The central agent should be stateless with respect to any single request beyond the request itself. Session context lives in a store the centre reads and writes but does not hold in memory across turns, because a stateful centre cannot be scaled horizontally without affinity, and affinity is the mechanism by which a routine deployment becomes a user-visible incident.

Long-running work sits behind a deliberate boundary. Anything that must survive a process restart, wait on a human decision, or run for longer than the conversational budget belongs in a durable execution engine invoked by the centre and owned separately. The distinction worth enforcing is between a journey and a turn: a turn has a latency budget of the kind divided above, and a journey may last days. Collapsing the two is the most common reason agent platforms cannot support approval steps without either blocking a request thread or losing state.

13.5 The single chokepoint, and why it is worth the risk

Concentrating classification, authority and composition in one component is a concentration of risk, and Section 11.4 states that cost without mitigation. It is accepted for one reason: a deterministic chokepoint is the only place where an authority decision can be made once, logged once, and reproduced later. Distributing that decision to domains means implementing it many times, which means implementing it differently, which means the answer to "was this permitted" becomes a survey rather than a lookup.

The mitigation is not to weaken the chokepoint but to make it boring: no model in the authority path, no dependency on discovery that cannot fall back to a cached last-known-good registry, no shared mutable state between requests, and a degraded mode that is exercised in production regularly rather than documented and left to rot.

14. The Model Tiering Ladder

14.1 Why tiering rather than one good model

The default architecture in most agent implementations is one capable model used for everything, on the reasonable grounds that a single model is simpler to operate, evaluate and reason about. The objection to it is not quality; it is that the distribution of work is extremely skewed, and a single model prices every turn at the cost of the hardest turn.

In a mature deployment the majority of turns are lookups with clear intent, unambiguous capability selection and a response that is mostly a template with values substituted in. Sending those turns through a large model buys nothing measurable and costs latency on the critical path, GPU capacity that could serve the hard turns, and money. Meanwhile the small minority of genuinely hard turns, which need cross-domain decomposition or contested policy interpretation, are exactly the turns that a small model answers confidently and wrongly. One model cannot be correctly sized for both populations. Tiering is the recognition that the correct model is a function of the turn.

TIER	WHAT IT IS FOR	LATENCY	SHARE
Tier 0 Deterministic no model at all <i>escalate only when the tier above cannot decide</i>	- exact and pattern intent match - cached and templated answers - rules, arithmetic, schema checks	0-5 ms per call	60-75% of turns of production turns
Tier 1 On prem, quantized, small 1-4B, FP8 or INT4, on prem <i>escalate only when the tier above cannot decide</i>	- task and risk classification - capability shortlisting - parameter extraction, rail scoring	15-40 ms per call	20-30% of turns of production turns
Tier 2 On prem, quantized, mid 8-32B, FP8, on prem <i>escalate only when the tier above cannot decide</i>	- multi-fact synthesis with citations - ambiguous single domain routing - output groundedness adjudication	120-400 ms per call	5-12% of turns of production turns
Tier 3 Frontier reasoning API or large on prem MoE <i>escalate only when the tier above cannot decide</i>	- cross domain decomposition - novel or contested policy reasoning - plan repair after failed attempts	0.8-4 s per call	1-4% of turns of production turns
Escalation is a decision with a cost, not a default			
Escalate when - classifier margin below threshold - two or more domains required - no capability matched the goal - previous attempt failed validation		Do not escalate when - the answer is a lookup - the tier below already agreed - only phrasing is uncertain - the budget is already spent	

Figure 7. The four tiers, with the work each is appropriate for, its per-call latency and the share of production turns it should absorb. The lower panel is the part that is usually missing from tiering discussions: an escalation policy that says when not to escalate is what keeps the ladder from collapsing into "always use the biggest model".

14.2 The four tiers

Table 11. The tiering ladder. Latency figures are per call and exclude queueing; share figures are the target distribution of production turns in a mature deployment, not a measurement of any specific system. Modelled targets.

Tier	What it is	Appropriate work	Per-call latency	Target share of turns
Tier 0	Deterministic code, no model	Exact and pattern intent match, cached and templated answers, rules, arithmetic, schema checks	0 to 5 ms	60 to 75%
Tier 1	On-prem quantized small model, 1 to 4B, FP8 or INT4	Task and risk classification, capability shortlisting, parameter extraction, rail scoring	15 to 40 ms	20 to 30%
Tier 2	On-prem quantized mid model, 8 to 32B, FP8	Multi-fact synthesis with citations, ambiguous single-domain routing, output groundedness adjudication	120 to 400 ms	5 to 12%
Tier 3	Frontier reasoning, API or large on-prem mixture of experts	Cross-domain decomposition, novel or contested policy reasoning, plan repair after a failed attempt	0.8 to 4 s	1 to 4%

The shares are targets, and treating them as such is the point: they are a design objective to be measured against, and a deployment where Tier 3 absorbs 20 percent of turns has either an unusually hard workload or, far more likely, a classifier that has not been tuned and an escalation policy that defaults to escalating.

Tier 0 deserves a defence because it is the tier most often skipped. A deterministic tier feels primitive next to a model that can handle phrasing variation, and it is the highest-leverage component in the system: it is free, it is instant, it never hallucinates, and it is trivially testable. "How many points do I have" is not a reasoning problem. The engineering task is to recognise it as such cheaply, which is a matching problem over a bounded set of intents plus a cache, and to escalate the moment matching is uncertain.

Design rule. Escalation is a decision with a cost, not a default. A turn moves up a tier only when the tier below can name why it could not decide, and that reason is recorded on the trace. "The model was available" is not a reason.

14.3 The escalation policy

An escalation policy needs both halves written down, and the second half is the one that gets omitted.

Escalate when the classifier's margin between its top two candidate intents is below a calibrated threshold; when two or more domains are required to answer, because composition is where small models produce plausible and wrong plans; when no capability matched the stated goal, because the correct output may be a decomposition into capabilities that do exist; when a previous attempt failed output validation, because plan repair is genuinely a reasoning task; or when the turn carries an irreversible effect and the plan's structure is not an exact match for a known-good shape.

Do not escalate when the answer is a lookup whose parameters were extracted with high confidence; when the tier below produced an answer that agreed with a deterministic check; when the only uncertainty is phrasing, which is a composition problem rather than a reasoning problem and belongs in a template; or when the remaining latency budget cannot accommodate the higher tier, in which case the correct behaviour is to answer at the current tier or return a named partial, not to blow the budget and hope.

That last clause is worth stating explicitly, because it is the difference between a budget and an aspiration. The escalation gate at stage 7 of Table 9 is a 2 ms deterministic check whose inputs include the time already spent. A system that escalates without consulting its remaining budget does not have a budget.

14.4 Confidence gating, and what a margin actually means

Escalation policies frequently gate on a model's reported confidence, which is a mistake worth naming. A language model's token probabilities are not calibrated probabilities of correctness, and a classifier fine-tuned on routing data is calibrated only to the distribution it was trained on. Three practices make gating defensible.

Use margin rather than absolute score. The gap between the top two candidates is a far better signal of ambiguity than the top score alone, and it is more stable across model updates.

Calibrate the threshold against a labelled routing set, and recalibrate whenever the model, its quantization format or the capability catalogue changes. A threshold inherited from a previous model version is an unmeasured constant.

Track the two error rates separately and price them differently. Escalating a turn that did not need it costs latency and money. Failing to escalate a turn that needed it produces a wrong answer, and if that turn carried a write, it produces a wrong action. The threshold should be set from the ratio of those costs, and that ratio is a business input rather than an engineering one.

14.5 The cost arithmetic that justifies the ladder

The following is modelled from the tier shares and latencies above, for one million turns, to show where the saving actually comes from. It uses relative cost units rather than currency, taking a Tier 1 call as one unit, Tier 2 as eight, and Tier 3 as ninety, which is the right order of magnitude for on-prem GPU seconds versus frontier API pricing but should be replaced with local figures.

Table 12. Modelled control-path cost and mean added latency per million turns, comparing a tiered ladder against a single-model deployment at each tier. The single-model rows are what the same traffic costs when every turn is priced at the cost of the hardest turn. Modelled from the shares in Table 11.

Configuration	Relative control-path cost	Mean added latency per turn	Note
Tiered ladder as specified	1.0x baseline	about 55 ms	68% of turns cost nothing at all
Everything at Tier 1	0.7x	about 30 ms	Cheaper and faster, and wrong on the 6 to 16% of turns that need synthesis or reasoning
Everything at Tier 2	5.6x	about 260 ms	Conversational budget consumed by the control path alone
Everything at Tier 3	63x	about 1,900 ms	Not a viable interactive system at any price

The second row is the important one, because it is the honest counter-argument to tiering: a single small model is cheaper and faster than a ladder. It is also unable to do the hard turns, and the failure is silent. The ladder's value is not that it minimises cost; it is that it minimises cost subject to keeping a capable model available for the turns that need one.

14.6 Operating the ladder

Three operational commitments make tiering work in practice rather than on a diagram.

Every tier is separately observable. The trace records which tier answered, why the tier below declined, the margin at the gate, and the latency contributed. Without this, the share distribution is unknown, and an unmeasured share distribution means the ladder cannot be tuned and its cost cannot be attributed.

Tier boundaries are policy, not code paths scattered through the runtime. The gate is one component with one configuration, so that changing when escalation happens is a configuration change with an evaluation suite attached, not a diff across the request path.

Each tier has a defined behaviour when the tier above is unavailable. If the frontier provider is unreachable, the system does not fail; it answers at Tier 2 with a recorded degradation, or returns a named partial. This is the

local-fallback discipline that Chapter 15 argues for at length, and it is the reason Tier 3 must never be the only path to any journey a customer depends on.

15. On-Prem Quantized Versus Frontier API

15.1 The question, stated so that it can be answered

The decision under examination is narrow, and most disagreements about it come from stating it too broadly. The question is not whether frontier models are better than small quantized ones; they plainly are, on the work that requires them. The question is not whether to use a frontier provider at all; this architecture uses one at Tier 3. The question is what should run on the control path that every single turn traverses, which in Table 9 means stages 3, 4, 6, 10 and 11.

Those stages have a specific character. They classify, shortlist, extract parameters, adjudicate groundedness and compose. They are high-volume, low-ambiguity, narrow-output tasks, and they are the tasks on which a well-tuned 3B model quantized to FP8 is close to indistinguishable from a frontier model. They are, in other words, precisely the stages where frontier capability buys the least and frontier latency costs the most.

15.2 The same journey, twice

The waterfall below holds the domain work constant at 180 ms and changes only the model serving the control path. It is modelled from the stage budget of Chapter 13, using 30 to 55 ms per on-prem Tier 1 or Tier 2 call in the same rack, and 210 to 900 ms per frontier API call inclusive of network round trip, provider queueing and generation. Those API figures are representative of a well-behaved commercial endpoint reached from a private network with connection reuse; they are not worst cases, and they do not include the retry that a p99 excursion often triggers.

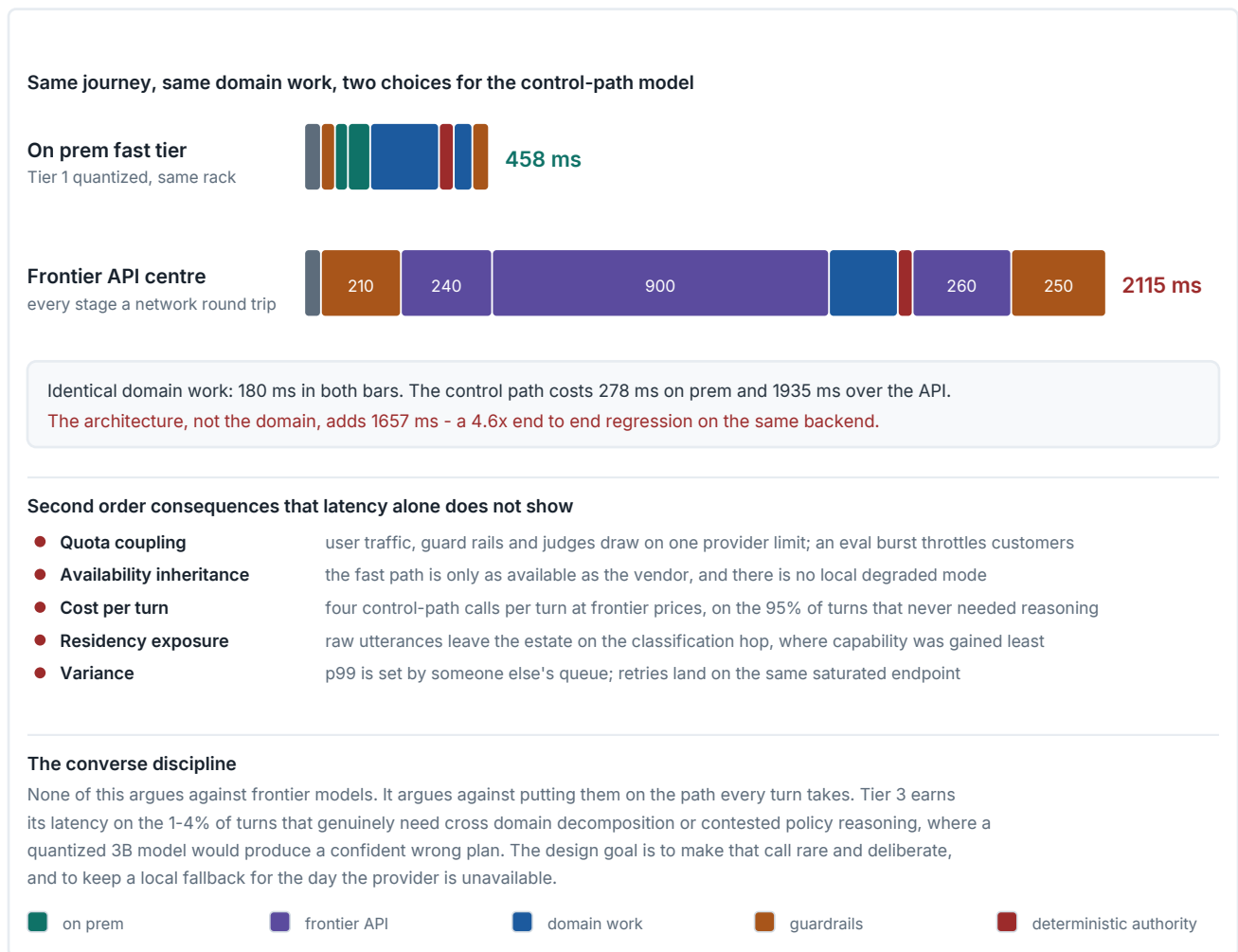


Figure 8. The same journey with the control path on-prem and on a frontier API. Identical domain work of 180 ms appears in both bars, so the entire difference is architectural. The lower panels are the part of the comparison that latency alone does not show, and the closing note states the converse discipline: this is an argument about placement, not about model quality.

Table 13. Stage-by-stage comparison of the same read-path journey with an on-prem quantized control path against a frontier-API control path. Domain execution and deterministic authority are identical in both columns by construction. Modelled from the assumptions stated above.

Stage	On-prem fast tier	Frontier API centre	Delta
Edge and identity	40 ms	40 ms	0 ms
Input rails	33 ms	210 ms	+177 ms
Classify and shortlist	30 ms	240 ms	+210 ms
Plan	55 ms	900 ms	+845 ms
Domain execution	180 ms	180 ms	0 ms
Authorize and validate	35 ms	35 ms	0 ms
Compose	45 ms	260 ms	+215 ms
Output rails	40 ms	250 ms	+210 ms
Total	458 ms	2,115 ms	+1,657 ms
Of which control path	278 ms	1,935 ms	7.0x

The end-to-end regression is a factor of 4.6, and the control-path regression is a factor of 7.0. The second figure is the honest one, because it isolates the part of the system the choice actually affects. A reader who believes the API figures are pessimistic should note that even at a uniform 120 ms per frontier call, four

control-path calls plus rails still put the journey above 1.1 s, which is outside a conversational budget before any domain has been slow.

15.3 Why the API penalty is structural rather than incidental

It is tempting to treat the gap as an engineering problem to be optimised away. Four components of it are not.

The round trip is a floor. A cross-region hop costs tens of milliseconds before the provider has done anything, and it is paid on every one of the four or five control-path calls, not once per turn.

Prefill is repaid on every turn. The control-path prompt includes instructions, the eligible capability shortlist and conversation context. On-prem, prefix caching is under your control and resident in your own KV cache. Through an API, caching behaviour is the provider's policy, varies by endpoint and version, and is not something you can size, pin or guarantee.

Queueing is someone else's. On-prem, a saturated router pool is a capacity problem you can see in your own metrics and fix by adding a card or shedding assurance load. On a shared endpoint, the queue ahead of your request contains other customers' traffic, and your only lever is a retry that lands on the same saturated endpoint. This is why the variance consequence in Figure 8 matters more than the mean: p99 is set by a system you do not operate.

Generation length is not free. Even a short classification response incurs the provider's time to first token plus its decode rate, and a control-path call that returns forty tokens over an API is not meaningfully cheaper than one returning four hundred, whereas on-prem the difference is real and measurable.

15.4 The second-order consequences

Latency is the most visible cost and not the most dangerous one. Four consequences deserve to be argued explicitly, because they are the ones that surface months after the architecture has been chosen.

Quota coupling turns assurance into an availability risk. If user traffic, guardrail scoring and LLM-as-judge evaluation all draw on the same provider quota, then an evaluation run competes with customers. This is the failure this architecture works hardest to prevent, because it inverts the relationship between assurance and service: the mechanism that exists to protect quality becomes the mechanism that degrades availability. Chapter 27 makes quota isolation a hard requirement for exactly this reason.

Availability is inherited rather than engineered. If classification is an API call, then a provider incident means the platform cannot classify, which means it cannot route, which means it cannot answer at all. The fast path's availability becomes the product of your own availability and the vendor's, and no amount of retry logic fixes a dependency that is down. On-prem Tier 1 with Tier 0 fallback fails differently: deterministic routing and templated answers continue to serve the majority of turns.

Cost per turn scales with traffic in the wrong currency. On-prem serving is a capital and power cost that is fixed with respect to turn count once provisioned, so the marginal turn is close to free until the pool saturates. API serving is a variable cost per call, paid four or five times per turn, on the large majority of turns that never needed reasoning. The crossover is worth computing rather than asserting, and it is computed below.

Data residency exposure is highest where capability gain is lowest. The classification hop sees the raw utterance, before any redaction that a later stage might apply, and that utterance may contain account numbers, health information or anything else a customer chose to type. Sending it out of the estate to decide whether a request is a balance lookup or a dispute is the worst available trade: maximum exposure, minimal capability benefit. If one argument in this chapter survives disagreement about every latency figure, it should be this one.

15.5 The crossover, computed

The following is modelled and offered as arithmetic to be redone with real quotes and real hardware costs. It assumes 2.6 control-path model calls per turn, an average of 700 input and 120 output tokens per control-path call, a blended frontier price of 0.60 dollars per million tokens for a small commercial model, and an amortised on-prem cost of 3.20 dollars per GPU hour inclusive of power, cooling, hosting and a three-year depreciation on the card.

Table 14. Modelled monthly control-path cost at three traffic volumes, comparing frontier API serving against on-prem quantized serving sized for the same load with headroom. The GPU counts come from Chapter 22. Replace every input with a local figure before using this to decide anything. Modelled.

Monthly turns	Control-path calls	Frontier API cost	On-prem fleet	On-prem cost	Ratio
500,000	1.3 M	about 640 dollars	1 GPU	about 2,300 dollars	0.28x, API cheaper
25,000,000	65 M	about 32,000 dollars	5 GPUs	about 11,500 dollars	2.8x, on-prem cheaper
400,000,000	1,040 M	about 512,000 dollars	40 GPUs	about 92,000 dollars	5.6x, on-prem cheaper

The shape of this table is the argument, not its values. At pilot volume the API is cheaper, and building a GPU fleet to serve half a million turns a month is capacity theatre. Somewhere in the low tens of millions of turns per month the fixed cost of a small fast-tier fleet is repaid, and past that the gap widens roughly linearly because API cost tracks volume while GPU cost tracks peak concurrency. This is also why the roadmap in Chapter 31 starts on APIs and moves the fast path in-house later: the correct sequence is to measure the workload with rented capacity, then buy hardware sized from measurement.

Decision. Start on frontier APIs for everything, including the control path, and instrument it thoroughly. Move the control path on-prem when any one of the following becomes true: control-path spend exceeds the amortised cost of the fleet that would replace it, p99 conversational latency is dominated by API round trips, a residency or contractual constraint forbids raw utterances leaving the estate, or an evaluation burst has throttled user traffic even once.

15.6 Where frontier models genuinely win

A chapter that only criticised the API path would be the caricature this document set out to avoid. Frontier models are the correct choice for Tier 3 work, and the reasons are specific.

They are substantially better at decomposing a goal into a plan when the goal spans domains and the correct sequence is not obvious. They are better at reasoning about policy language that contradicts itself or does not cleanly cover the case at hand, which is common in real regulatory text. They are better at plan repair, meaning the second attempt after an output validation failure, where the useful behaviour is to reconsider the approach rather than retry the same call. And they require no capacity planning, no quantization work, no serving expertise and no hardware, which at Tier S is not a minor consideration but the deciding one.

The design that follows from this is a ladder whose top rung is rented, whose lower rungs are owned, and whose escalation policy is tuned to make the rented rung rare. That is a different position from either "run everything locally" or "call the best model every time", and it is the position this document defends.

15.7 What would falsify this chapter

The argument for an on-prem fast path fails if any of the following can be shown on your own workload. That a frontier endpoint reachable from your network sustains a p99 under about 120 ms for a short classification call, including retries, over a full week of production traffic. That your control-path spend at target volume remains below the amortised cost of the equivalent GPU fleet. That the provider's caching, quota and availability behaviour can be contracted to a level that removes the coupling and inheritance arguments. Or that no raw utterance reaching the classification hop can contain regulated data, which is a claim about your input filtering rather than about your users.

Where those hold, the API-centred control path is simpler, cheaper to operate and requires none of the capabilities in Chapter 25. That is a real advantage and the honest recommendation for a large fraction of deployments, particularly at Tier S.

16. Guardrails: Taxonomy and Placement

16.1 What a guardrail is, and what it is not

A guardrail is a control that constrains what enters or leaves a model-mediated boundary, evaluated independently of the model whose behaviour it constrains. The independence is the load-bearing part. An instruction in a system prompt telling a model not to reveal account numbers is not a guardrail; it is a request, evaluated by the same component whose failure it is supposed to catch, and it fails in exactly the circumstances where a control is needed.

Three distinctions are worth fixing before the taxonomy, because conflating them produces most guardrail design errors.

A guardrail is not an authority check. Entitlement, limits and write permission are decided by deterministic code reading structured state, as in stage 9 of Table 9. Rails constrain content and shape. Authority constrains effect. A system that uses a rail to prevent an unauthorised transfer has made a category error, because rails are probabilistic and authority must not be.

A guardrail is not an evaluation. Rails run inline on every request and must answer within a latency budget. Evaluations run against a corpus and may take as long as they need. The two are frequently confused because both produce verdicts about quality, and the consequence of the confusion is either an evaluation on the critical path, which destroys latency, or a rail treated as sufficient evidence of quality, which is the mistake that step 7 of the reference implementation exists to illustrate.

A guardrail is not a substitute for a bounded action space. The strongest control over what an agent can do is what it is wired to be able to do. A rail that inspects a tool call for suspicious parameters is a weaker control than a tool that is not callable in this context at all, and the closed-world reachability check of Section 4.3 is worth more than any inline inspection.

16.2 The four boundaries where rails attach

Rails belong at trust boundaries, and there are exactly four in this architecture. Placing them anywhere else produces controls that are either redundant or unenforceable.

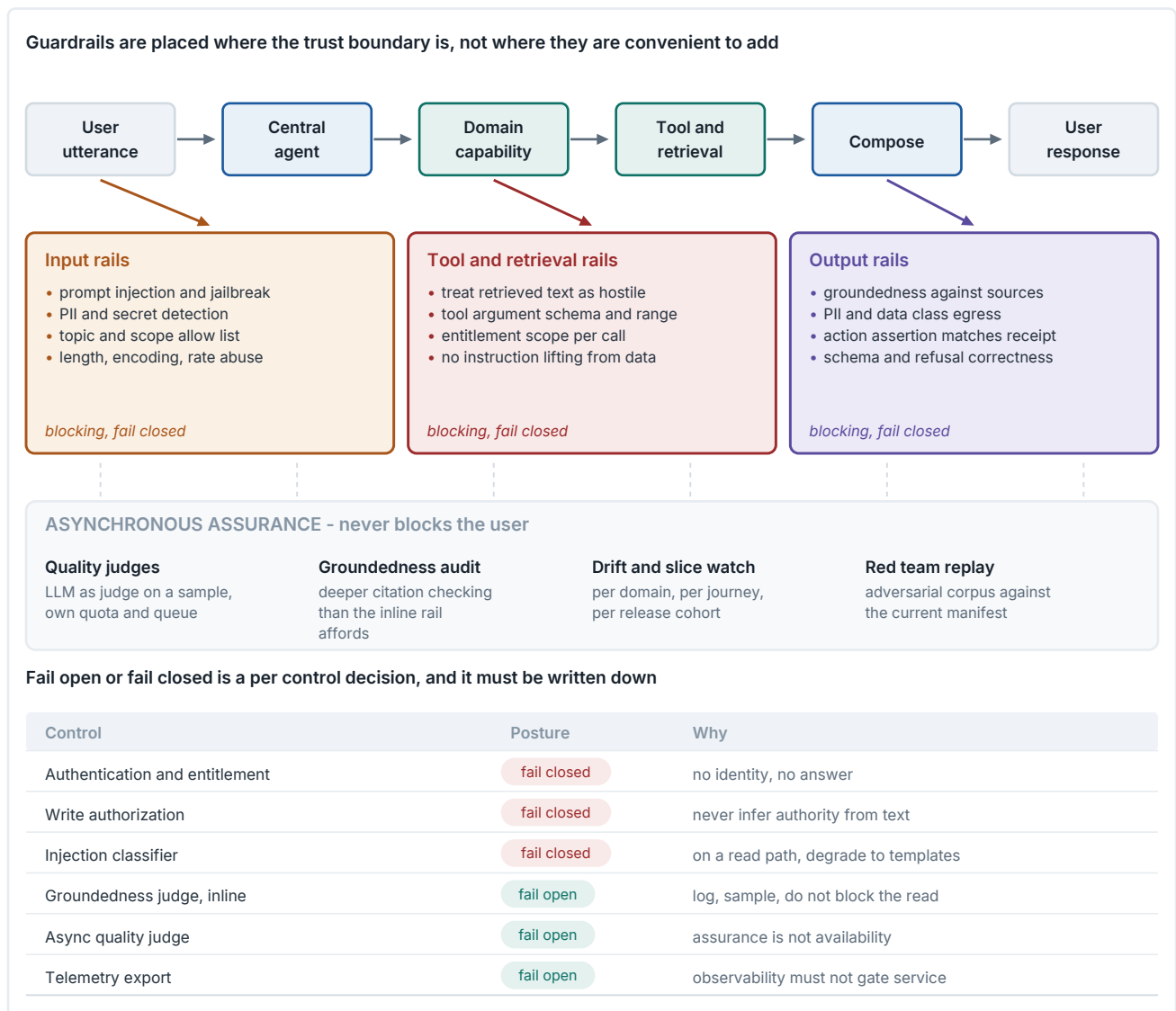


Figure 9. Where rails attach, and what each set is responsible for. The middle band is the one most often missing: retrieved documents and tool output cross a trust boundary and must be treated as hostile input, not as trusted context. The lower band is asynchronous assurance, which is deliberately never allowed to block a user, and the table states the fail-open or fail-closed posture per control.

The first boundary is between the user and the centre, where the input rails sit. The second is between the centre and a domain capability, where the tool argument and entitlement scope checks sit. The third is between a tool or retrieval source and the model that will consume its output, which is the boundary most implementations omit. The fourth is between composition and the user, where output rails sit.

A useful test of whether a control is at a boundary: ask what the control protects against, and then ask whether the thing it protects against can reach the protected component by another route. If it can, the control is decorative.

16.3 The input rail taxonomy

Table 15. Input rails, with the mechanism, the tier that implements it and where in the stage budget it sits. Latency figures are per-check contributions to stages 2 and 3 of Table 9. Rows marked Tier 1 are model calls; the rest are deterministic.

Rail	What it catches	Mechanism	Tier	Budget
Length, encoding and rate	Oversized payloads, encoding smuggling, abuse patterns	Deterministic limits and normalisation	Tier 0	under 1 ms
Secret and credential detection	API keys, tokens and passwords pasted into a chat	Pattern and entropy detection	Tier 0	1 to 3 ms
PII detection and redaction	Regulated data entering logs, prompts or a third-party endpoint	Named-entity and pattern DLP	Tier 0 or 1	2 to 8 ms
Topic and scope allow list	Requests the platform has no business answering	Classifier against declared scope	Tier 1	10 to 20 ms
Prompt injection and jailbreak	Attempts to override instructions or extract the prompt	86M-class classifier, escalating to a guard model	Tier 1, then 2	20 to 50 ms
Harm and hazard categories	Content the organisation has undertaken not to process	Guard model in no-think mode	Tier 2	40 to 90 ms
Identity and session integrity	Replayed, forged or expired sessions	Deterministic verification	Tier 0	included in stage 1

Two rows deserve comment. PII redaction is placed early because everything downstream, including logs, traces, prompts and any third-party call, inherits whatever it fails to catch, which is the residency argument of Section 15.4 restated as a control. And the injection rail is the only input rail permitted to escalate to a larger model, because it is the one whose false negatives are actively adversarial rather than accidental.

16.4 Injection usually does not arrive in the user's message

The single most common gap in guardrail implementations is that all the effort goes on the user's utterance, which is the input an attacker controls least conveniently, and none on the text a tool or a retrieval source returns.

Consider the mechanism concretely. A domain capability returns a record containing a free-text field, perhaps a merchant description, a support note or a document chunk. That text is inserted into the composition prompt so the model can summarise it. If it contains an instruction, the model has no reliable way to distinguish it from the platform's own instructions, because both arrive as tokens in the same context. The attacker does not need access to the conversation at all; they need to have written text into a system that the platform later retrieves. In a retrieval-augmented deployment this includes any document corpus users can contribute to, which is most of them.

Four controls address this, and none of them is a prompt instruction.

Structure the boundary rather than relying on the model to respect it. Domain results are passed as structured fields with declared types, and free-text fields are marked as data. Composition prefers templates with substituted values over free generation, which is why stage 10 of Table 9 is labelled "structured template where possible" and is priced at 45 ms rather than at the cost of a generation.

Scan tool and retrieval output with the same injection rail used on user input, because the threat is identical and only the delivery route differs.

Never allow a tool result to expand the action space. If retrieved text names a capability, that is not a request the planner may honour; capability selection happens before composition and from the registry only. This is the property closed-world reachability makes checkable.

Bound the effect of composition. If composition can only emit text, injected instructions can at worst produce a misleading answer, which the output rails then examine. If composition can trigger a call, injected instructions

can produce an action, and no output rail will save you.

Anti-pattern. Instructing the model to ignore instructions found in retrieved content. This is a request made to the component under attack, evaluated by the component under attack, in the same channel as the attack. It reduces the success rate of naive attempts, which makes it worse than useless, because it produces the appearance of a control and the measurement of its absence.

16.5 The output rail taxonomy

Output rails are the less developed half of most implementations, which is unfortunate, because they are the half that decides whether a wrong answer reaches a customer.

Table 16. Output rails, with what each one compares against. The third column is the important one: a rail that has nothing to compare against is a rail that is asking a model for an opinion.

Rail	What it catches	Compared against	Tier	Budget
Schema and contract	Malformed or unparseable responses, missing required fields	The declared response contract	Tier 0	under 1 ms
Groundedness	Claims not supported by any retrieved or returned evidence	The actual domain results in this turn	Tier 1 or 2	20 to 60 ms
Action assertion	A response claiming an effect that did not occur	The receipt or confirmation from the domain	Tier 0	1 to 3 ms
PII and data class egress	Regulated data leaving in a channel not cleared for it	The data classification of the destination channel	Tier 0	2 to 6 ms
Refusal correctness	Refusing for an unstated reason, or answering something that should have been refused	The recorded rail decision and reason code	Tier 0	under 1 ms
Numeric and unit sanity	Impossible balances, mismatched currencies, sign errors	Domain-declared ranges and units	Tier 0	1 to 2 ms
Tone and disclosure policy	Missing mandated disclosures, prohibited advice framing	Policy templates for this journey class	Tier 0 or 1	5 to 20 ms

The action assertion rail is the one to build first if only one output rail can be afforded. It is deterministic, it is cheap, and it catches the class of error with the worst consequences: a response that tells a customer something happened when it did not. Its implementation is a comparison between the claimed effects in the composed response and the receipts actually returned by the authority stage, which is possible only because authority is deterministic and produces receipts. This is an example of the general pattern in this architecture, where a deterministic guarantee at one stage makes a cheap check possible at a later one.

Groundedness is the rail most often over-scoped. It should ask whether the claims in this response are supported by the evidence returned in this turn, which is a bounded comparison. It should not ask whether the answer is correct in the world, which is unbounded, unanswerable inline, and belongs to domain evaluation. The wrong answer in step 7 of the reference implementation was perfectly grounded: the transaction was real, the amount was right, the arithmetic was wrong for a domain reason. No groundedness rail catches that, and none should be blamed for it.

16.6 Fail open or fail closed, decided per control

Every rail needs a documented posture for the case where it is unavailable, times out, or returns an error. The common mistakes are to choose one posture globally, or worse, to leave it as whatever the framework's default happens to be.

Table 17. Fail posture per control, with the reasoning. The pattern is that controls protecting authority and irreversible effects fail closed, and controls protecting quality fail open with a recorded degradation.

Control	Posture	Reasoning
Authentication and entitlement	Fail closed	No identity, no answer. There is no acceptable degraded mode for authority
Write authorization	Fail closed	Authority is never inferred from text, and an unavailable check is not an approval
Injection classifier, read path	Fail closed to a safe path	Degrade to deterministic routing and templated answers rather than refusing outright
Injection classifier, write path	Fail closed	An unscreened input must not reach a stage that can cause an effect
PII egress rail	Fail closed	The cost of a leak is asymmetric and irreversible
Action assertion	Fail closed	If the receipt cannot be checked, the claim must not be made
Groundedness rail, inline	Fail open, logged and sampled	Blocking reads on an unavailable quality check trades a large availability loss for a small quality gain
Asynchronous quality judges	Fail open	Assurance is not availability, and a judge outage must never be user-visible
Telemetry and trace export	Fail open	Observability must not gate service, however tempting it is to make it mandatory

The pattern generalises to a rule that is easier to apply than a table: fail closed where the failure is irreversible or where authority is involved, and fail open with a recorded degradation where the failure is a quality risk on a reversible read. What must never happen is an undocumented posture, because the default that a framework chooses under load is discovered during an incident.

17. The Guardrail Cascade

17.1 The guardrail tax, and why chaining safety calls does not work

The straightforward way to implement thorough guardrails is to run every check on every request. It is also the way to destroy the latency budget, and the arithmetic is unforgiving. A guard model at 65 ms, a topic classifier at 20 ms, a groundedness adjudicator at 45 ms and a PII model at 25 ms is 155 ms of rail work on a 460 ms budget, before the platform has classified the request or a domain has done anything.

What makes this worse than a simple latency problem is the behaviour it induces. Once rails consume a third of the budget, someone is asked to make the system faster, and rails are the easiest thing to remove because their contribution is invisible when they are working. Rails then get disabled selectively, usually without a record of which ones and for which journeys, and the system arrives at the worst state available: the cost of having built controls, without the coverage.

Anti-pattern. The guardrail tax. Chaining every safety model on the critical path of every request, then quietly disabling the ones that hurt most when latency becomes a complaint. The result is a control catalogue that does not describe the running system, which is more dangerous than having no catalogue, because it is trusted.

17.2 The cascade

The alternative is a cascade in which every turn pays for the cheapest checks, and only the turns that look ambiguous pay for the expensive ones. This is the same escalation logic as Chapter 14 applied to safety rather than capability, and it works for the same reason: the distribution of difficulty is skewed.

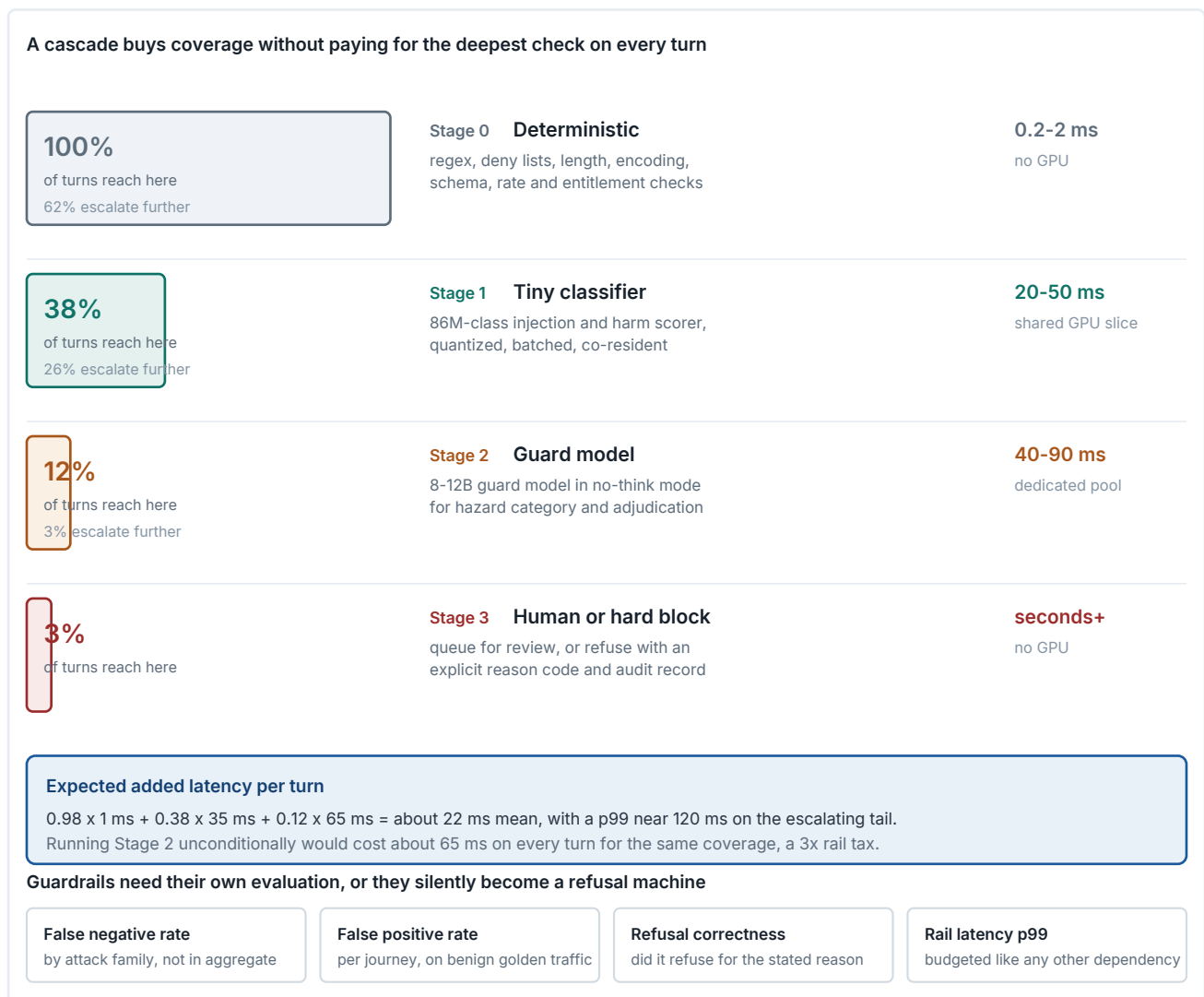


Figure 10. The cascade, with the share of turns reaching each stage, the latency each stage contributes and the hardware it needs. The arithmetic panel is the argument: the same coverage that would cost 65 ms on every turn costs about 22 ms on average, because the deepest check only runs on the 12 percent of turns that reached it. The lower band is the part that keeps the cascade honest, which is that the rails themselves are evaluated.

Table 18. The cascade stages, with the modelled share of turns reaching each, its latency contribution and the decision it makes. Shares are targets for a mature deployment and will differ substantially on adversarial or heavily regulated traffic. Modelled, with third-party latency figures noted in Table 19.

Stage	Check	Share of turns reaching it	Latency when it runs	Hardware	Escalates when
0	Deterministic: regex, deny lists, length, encoding, schema, rate and entitlement	100%	0.2 to 2 ms	none	A pattern matches, or the input is not obviously benign
1	Tiny classifier: 86M-class injection and harm scorer, quantized and batched	38%	20 to 50 ms	Shared GPU slice	Score exceeds a calibrated threshold, or margin is narrow
2	Guard model: 8 to 12B in no-think mode, hazard category adjudication	12%	40 to 90 ms	Dedicated pool	The category is one requiring human judgement, or confidence is low
3	Human review or hard block with a reason code	3%	seconds or longer	none	Terminal

The expected added latency is the share-weighted sum: about 1 ms on every turn, 35 ms on the 38 percent that reach stage 1, and 65 ms on the 12 percent that reach stage 2, which is about 22 ms of mean rail latency with a p99 near 120 ms on the escalating tail. Running stage 2 unconditionally would cost about 65 ms on every turn for the same coverage, which is roughly three times the rail tax for no additional protection.

Two design constraints keep this from degrading in practice. Stage 0 must be genuinely cheap, which means no model, no network call and no database lookup that is not already cached. And the escalation thresholds must be calibrated against measured traffic, because a stage 1 threshold that sends 80 percent of turns to stage 2 has quietly rebuilt the unconditional design while retaining the complexity of the cascade.

17.3 Model choices at each stage

The figures below are third-party published characteristics of guard models available at the time of writing, offered to make the cascade concrete rather than as a recommendation. Latency depends on input length, batch size, quantization and hardware, and every one of them should be re-measured with the recipe in Chapter 23.

Table 19. Representative guard models by cascade stage, with the size class and published purpose. All figures third-party published; latencies assume short inputs on a modern data-centre GPU with FP8 or equivalent quantization and batching, and are not vendor guarantees.

Stage	Model class	Size	Purpose as published	Indicative latency
1	Llama Prompt Guard 2	86M	Prompt injection and jailbreak detection on user and tool text	20 to 50 ms
1	Granite Guardian HAP	38M	Hate, abuse and profanity detection where the budget is tightest	under 20 ms
2	Llama Guard 4	12B	Multimodal hazard taxonomy classification for input and output	40 to 90 ms
2	Granite Guardian 4.1	8B	Harm, groundedness and jailbreak checks, with a no-think mode for latency	40 to 90 ms
0 to 2	NeMo Guardrails	orchestration	Rail definition, dialogue and flow constraints, and rail sequencing	20 to 80 ms of orchestration overhead

Two selection notes matter more than the specific models, because the models will change. First, prefer a small purpose-trained classifier over a general model with a safety prompt at stage 1: it is one to two orders of magnitude cheaper, it can be batched aggressively, and its failure modes are measurable on a labelled set. Second, use a guard model's no-think or non-reasoning mode at stage 2 unless the escalation genuinely needs

deliberation, because reasoning traces are the difference between 60 ms and several seconds.

17.4 The rails need their own evaluation

An unevaluated guardrail becomes a refusal machine. The failure is gradual: thresholds are tightened after an incident, nobody measures the effect on benign traffic, and the system slowly begins refusing legitimate requests. Because refusals look safe, the drift is invisible in the metrics teams usually watch.

Four measurements are the minimum, and they are properties of the rails rather than of the platform.

False negative rate, by attack family and not in aggregate. An aggregate number is dominated by whichever family is best represented in the test set, which is usually the easiest. Break it out by direct instruction override, indirect injection through retrieved content, encoding and obfuscation, multi-turn escalation, and tool-output smuggling. A rail can be excellent at the first and useless at the second.

False positive rate on benign golden traffic, per journey. Sampled from real production traffic that a human has confirmed benign, and reported per journey, because a rail that costs 0.5 percent of traffic overall may be costing 8 percent of one journey and making it unusable.

Refusal correctness. When the system refuses, did it refuse for the reason it stated, and was that reason a real policy? This catches the common failure where a rail fires for one reason and the response cites another, which destroys the audit trail and confuses the customer.

Rail latency at p99, budgeted like any other dependency. Rails are a dependency of the request path and should have a per-stage budget, a timeout, a circuit breaker and a documented fail posture from Table 17. A rail without a timeout is an availability risk wearing the costume of a control.

17.5 Who owns the thresholds

Thresholds are policy, and policy has an owner. The arrangement that works is that the platform team owns the mechanism, the cascade shape and the latency budget; risk or security owns the acceptable false negative rate by attack family; and the domain owns the journey-level false positive tolerance, because the domain is accountable for whether its customers can complete their task.

The reason to name owners explicitly is that threshold changes are the highest-leverage and least-reviewed changes in the system. Moving one number can shift a tenth of production traffic from stage 1 to stage 2, change mean latency by tens of milliseconds, and alter the refusal rate of a journey, with no code change and often no evaluation. Threshold changes therefore belong in the change types of Figure ? that trigger a required evaluation suite, and the reference implementation's change-classification rule treats a guardrail threshold change as a first-class change type for exactly this reason.